Cloud Background

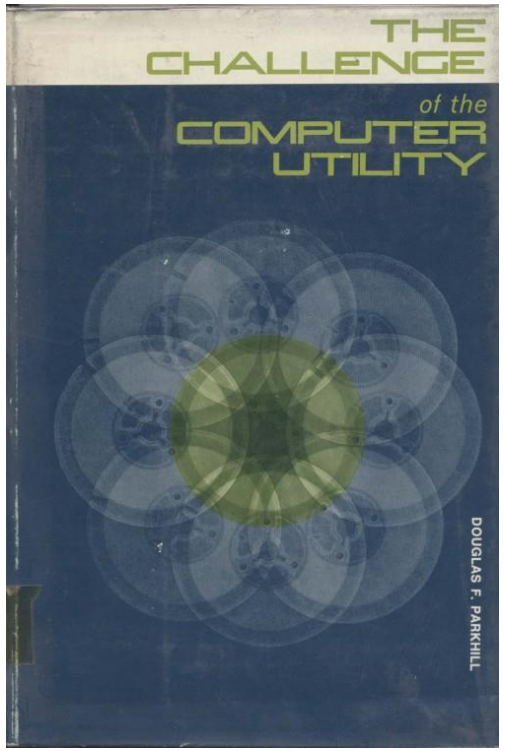
Course: Networking Principles in Practice – Cloud Networking
Module: Cloud Background



University of Colorado **Boulder**

Utility Computing

- You can rent computing resources by the hour / minute / second



1966

AWS Origins

- Amazon founded in 1994
- Originally, an online marketplace for books
- Broader e-commerce site



https://techcrunch.com/2016/07/02/andy-jassys-brief-history-of-the-genesis-of-aws/

AWS Origins (2)

- Around 2000, wanted to launch merchant.com, to help 3rd party merchants build online shopping sites
- But, it was a challenge to separate the various services to make a dev platform that would be useful for 3rd parties
- Set out to untangle that mess into a set of well-documented APIs.



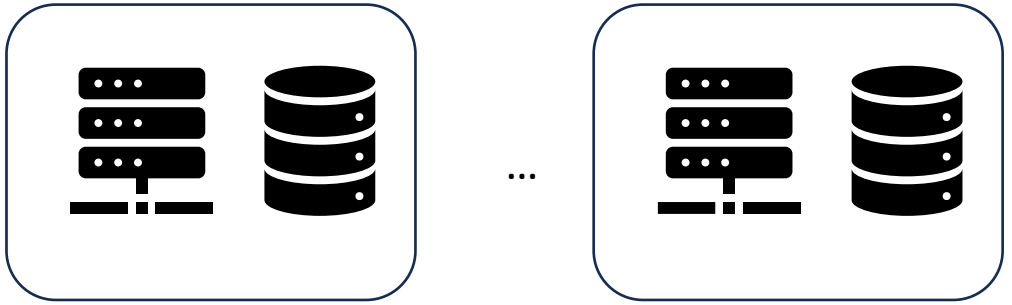
M&S

https://techcrunch.com/2016/07/02/andy-jassys-brief-history-of-the-genesis-of-aws/

AWS Origins (3)

- The executive team expected a project to take three months, but it was taking three months just to build the database, compute or storage component.

➔ The internal teams at Amazon then required a set of common infrastructure services everyone could access without reinventing the wheel every time



https://techcrunch.com/2016/07/02/andy-jassys-brief-history-of-the-genesis-of-aws/

AWS Origins (4)

- They realized they had also become quite good at running infrastructure services like compute, storage and database
- They had become highly skilled at running reliable, scalable, cost-effective data centers out of need.



https://techcrunch.com/2016/07/02/andy-jassys-brief-history-of-the-genesis-of-aws/

What is Cloud Computing?

(1) Elimination of an up-front commitment

- Previously, when wanted to launch a service:
  - Acquire some colocation space (which provides space, power, and network)
  - Purchase some equipment (servers, routers, firewalls)
  - Rack and stack the equipment
  - Configure the infrastructure

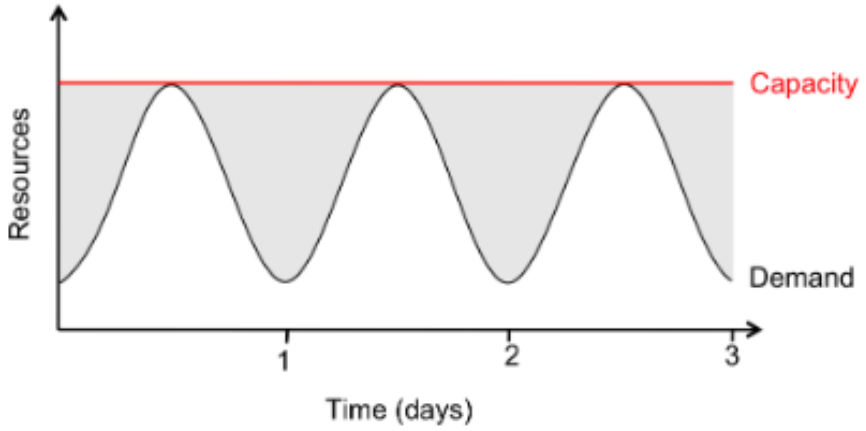
Capital expenditures

Time and money to build it out

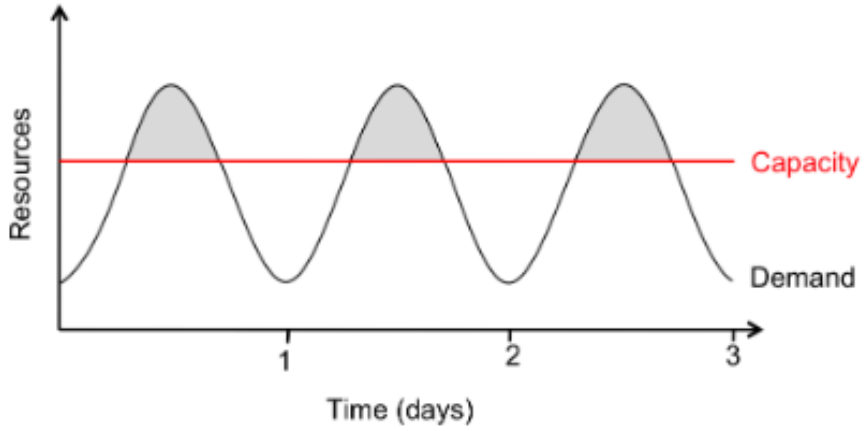
https://www2.eecs.berkeley.edu/Pubs/TechRpts/2009/EECS-2009-28.pdf

(2) Pay for use on a short-term basis

- With fixed capacity, result will be under or over provisioning
- Elasticity allows for provisioning for what's needed in the moment



(a) Provisioning for peak load



(b) Underprovisioning 1

https://www2.eecs.berkeley.edu/Pubs/TechRpts/2009/EECS-2009-28.pdf

(3) Illusion of Infinite Computing Resources

- Predicting the future is hard… will my service grow rapidly, slowly.
- Don’t want to hurt growth if didn’t plan for growth
  - Hiring is hard, provisioning takes time, etc.



Why it works

Hyper scale: Costs

- Costs of electricity, network, etc. (economies of scale)

Table 2: Economies of scale in 2006 for medium-sized datacenter (≈1000 servers) vs. very large datacenter (≈50,000 servers). [24]

Technology	Cost in Medium-sized DC	Cost in Very Large DC	Ratio
Network	$95 per Mbit/sec/month	$13 per Mbit/sec/month	7.1
Storage	$2.20 per GByte / month	$0.40 per GByte / month	5.7
Administration	≈140 Servers / Administrator	>1000 Servers / Administrator	7.1

https://www2.eecs.berkeley.edu/Pubs/TechRpts/2009/EECS-2009-28.pdf

Why it works: Expertise

- Recall AWS Origins…

…quite good at running infrastructure services like compute, storage and database

…highly skilled at running reliable, scalable, cost-effective data centers

Infrastructure as a Service

- Delivers on-demand infrastructure resources to organizations via the cloud, such as compute, storage, networking, and virtualization.
- Customers don’t have to manage, maintain, or update their own data center infrastructure, but are responsible for the operating system, middleware, virtual machines, and any apps or data.

https://cloud.google.com/learn/paas-vs-iaas-vs-saas

Platform as a Service

- delivers and manages all the hardware and software resources to develop applications through the cloud.
- Developers and IT operations teams can use PaaS to develop, run, and manage applications without having to build and maintain the infrastructure or platform on their own.

- Available from the cloud providers and from third parties (creating a really rich ecosystem)

https://cloud.google.com/learn/paas-vs-iaas-vs-saas

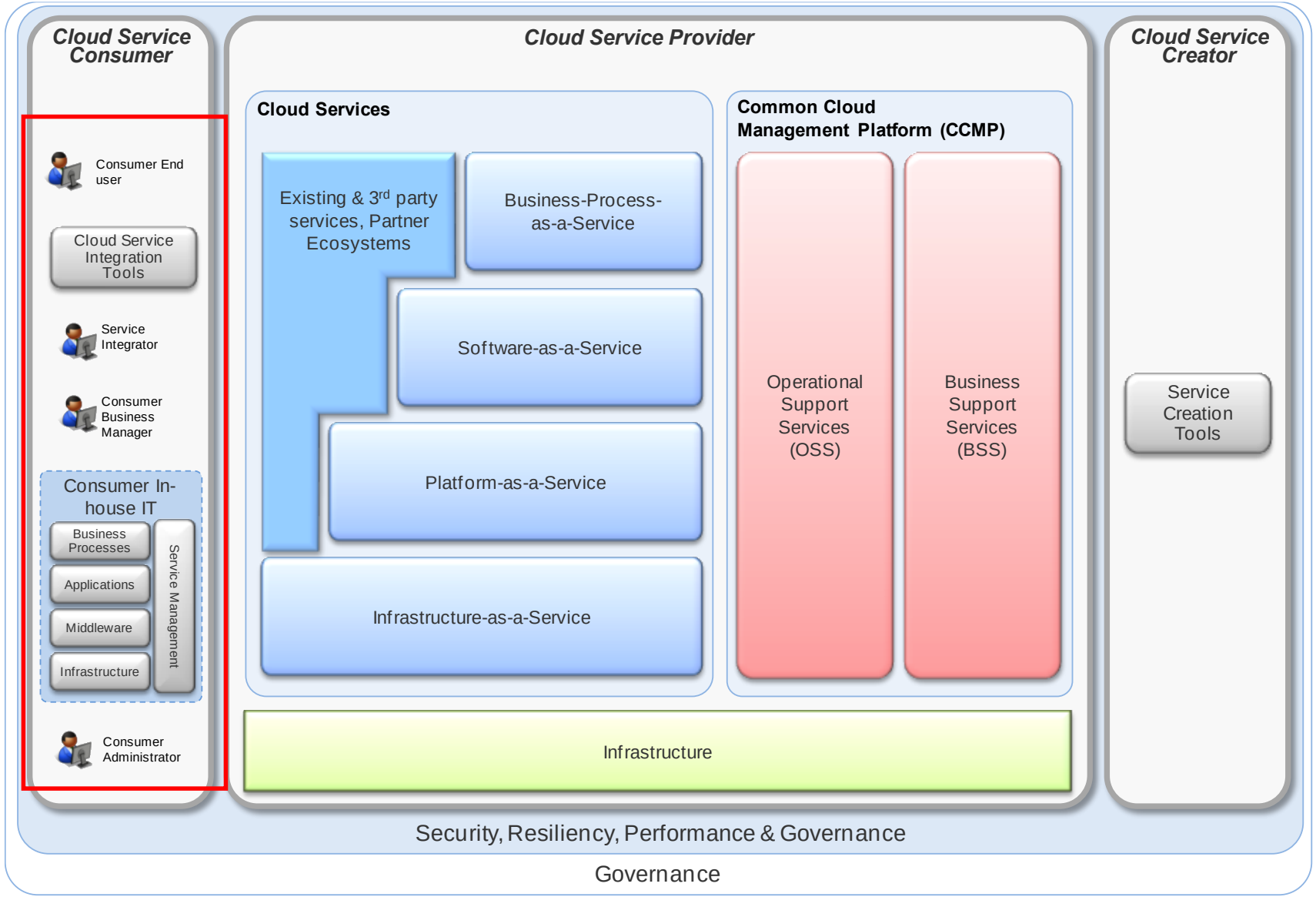
Software as a Service

- Provides the entire application stack, delivering an entire cloud-based application that customers can access and use.

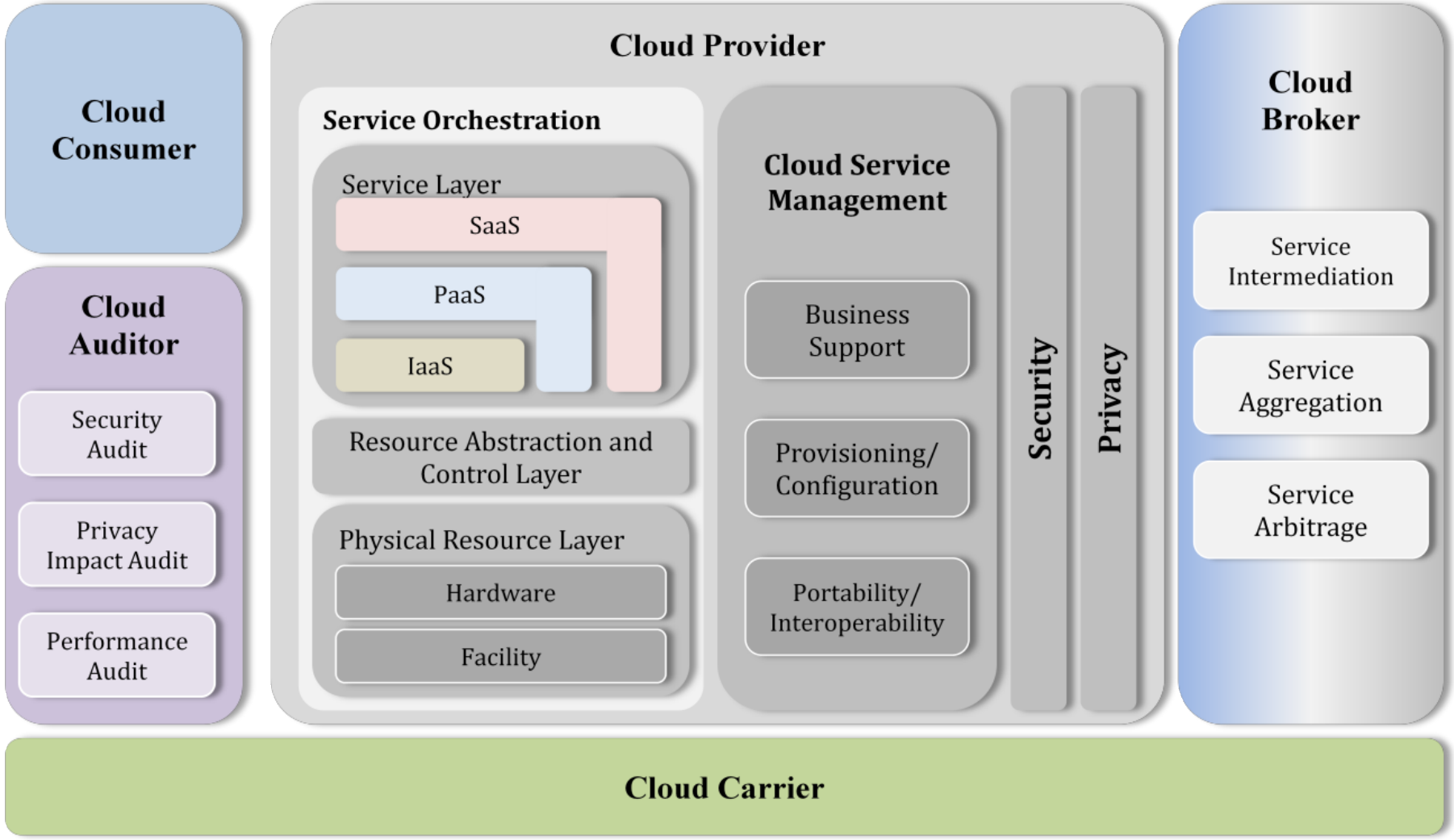
- Available from the cloud providers and from third parties (creating a really rich ecosystem)

https://cloud.google.com/learn/paas-vs-iaas-vs-saas

XaaS Ecosystem (from IBM)



NIST Cloud Computing Components



Summary of Benefits

- Trade capital expense for variable expense
- Benefit from massive economies of scale
- Stop guessing about capacity
- Increase speed and agility
- Focus on what matters
- Go global in minutes

https://digitalcloud.training/a-brief-history-of-aws-and-how-computing-has-changed/

Module Overview

- Virtualization Overview
- Datacenter Fabric (VL2)
- Google Cloud Platform
- Terraform Overview
- Terraform with GCP



University of Colorado **Boulder**

Virtualization Overview

Course: Networking Principles in Practice – Cloud Networking
Module: Cloud Background



University of Colorado **Boulder**

Problem

- Enterprise IT centers support many service applications
  - Microsoft Exchange
  - Oracle
  - SAP
  - Web servers
  - Citrix
  - …
- Each service application demands its own environment
  - Specific version of operating system
  - Multiple processors and disks
  - Specialized configurations
  - …

Problem (cont)

- Combining services on same server host is difficult (at best)
  - Conflicting demands
  - Incompatible loads
  - …
- Upgrading or commissioning a service is very difficult
  - Shadow server machines for debugging & testing
  - Complicated changeover tactics

Solving Configuration Management — Virtualization

- Decouple [OS, service] pair from hardware
- Multiplex lightly-used services on common host hardware
- Migrate services from host to host as needed
- Introduce new [OS, service] pairs as needed
  - Commissioning new services
  - Testing upgrades of existing services
  - Experimental usage

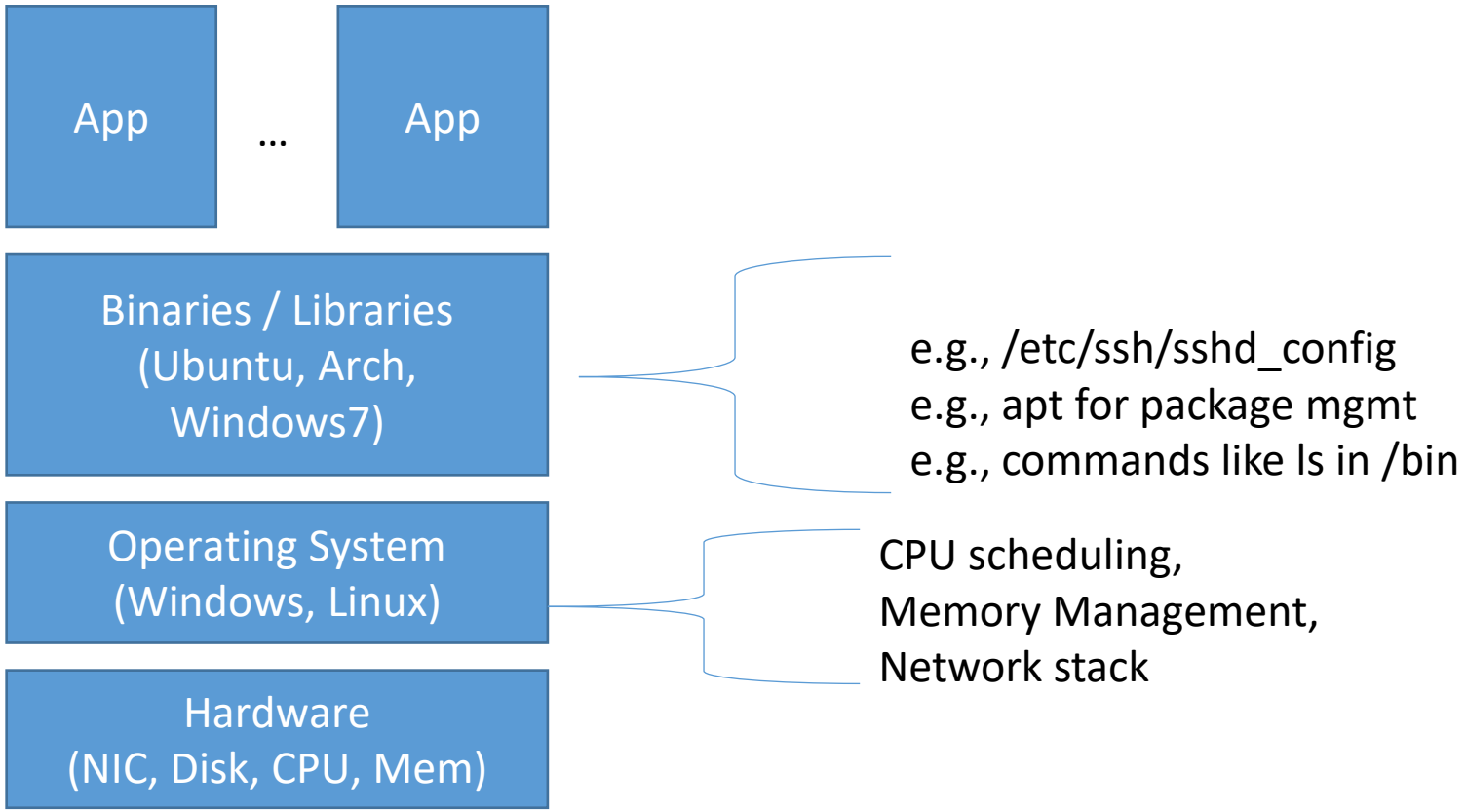
Virtual Machine

- A virtual machine provides interface identical to underlying bare hardware
  - i.e., all devices, interrupts, memory, page tables, etc.

- Virtual Machine Operating System (Hypervisor) creates illusion of multiple processors
  - Each capable of executing independently
  - No sharing, except via network protocols
  - Clusters and SMP can be simulated

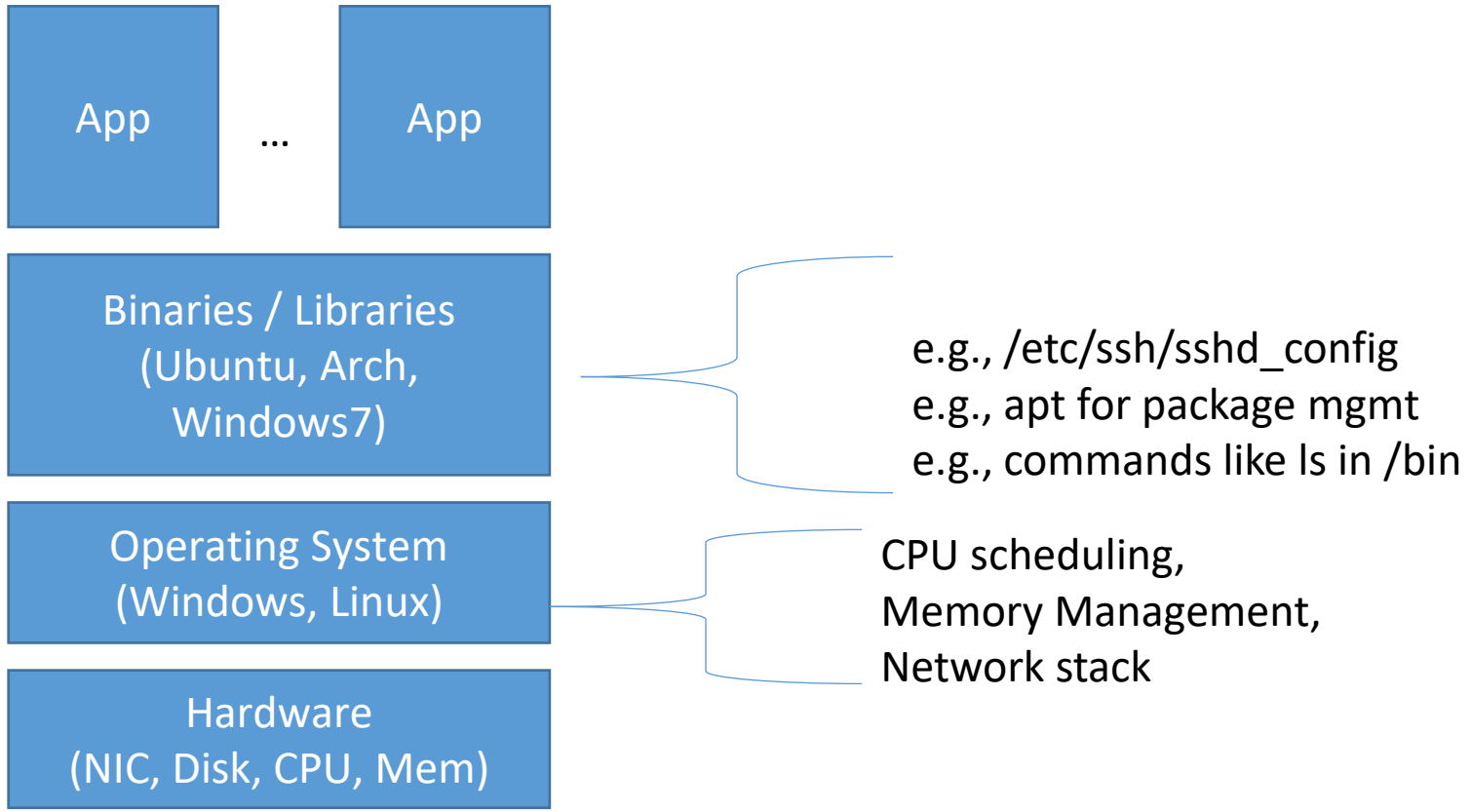
Bare Metal

Run applications directly on servers



Bare Metal

Run applications directly on servers



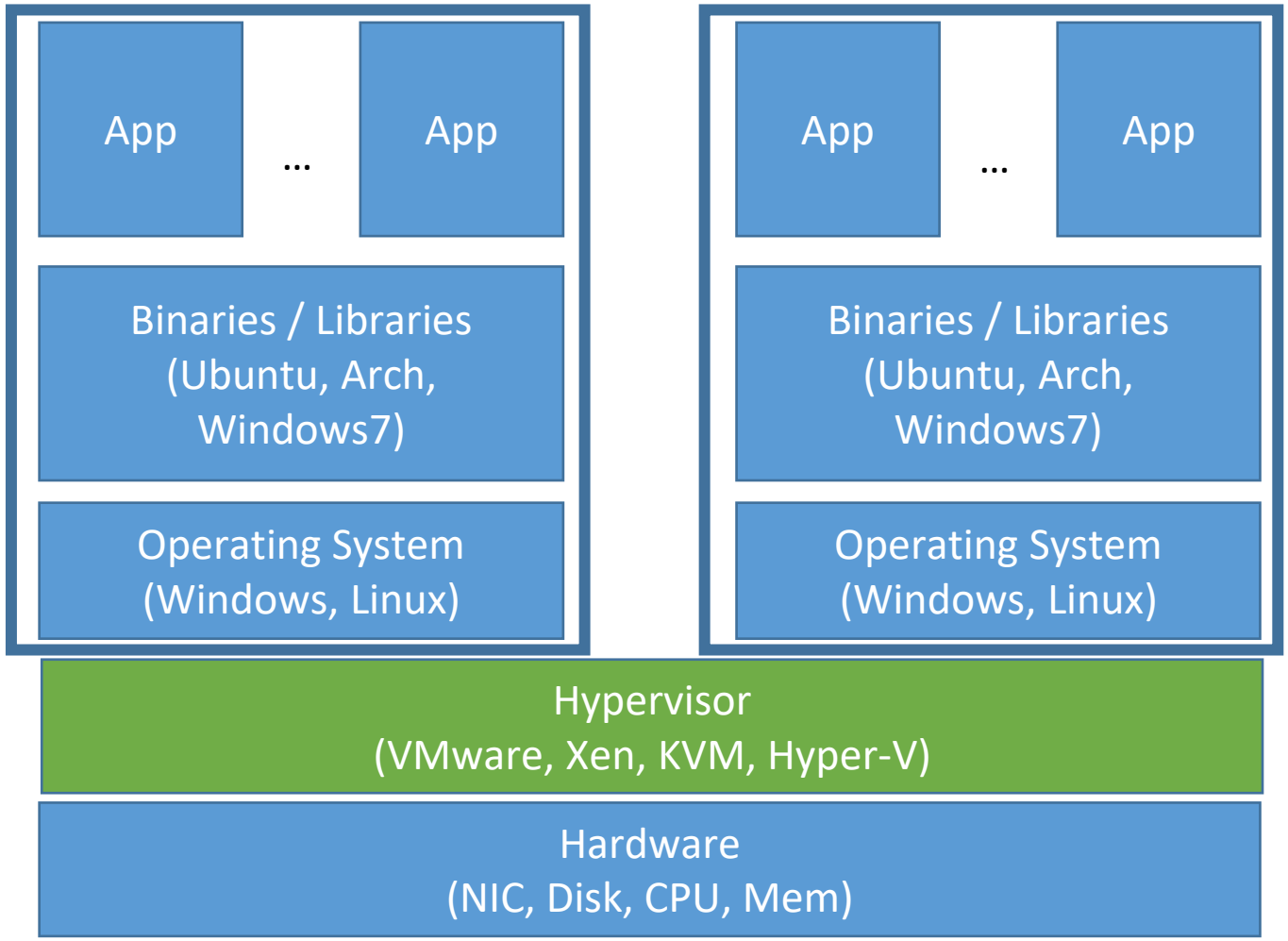
Challenges:

- Resource wastage (using only part of a server), so want to run multiple applications on the same server

- Applications might need different Operating Systems
- Applications might have variable workloads
- Applications might have conflicting dependencies (e.g., App1 needs python3, App2 needs python2) - sometimes complex to resolve

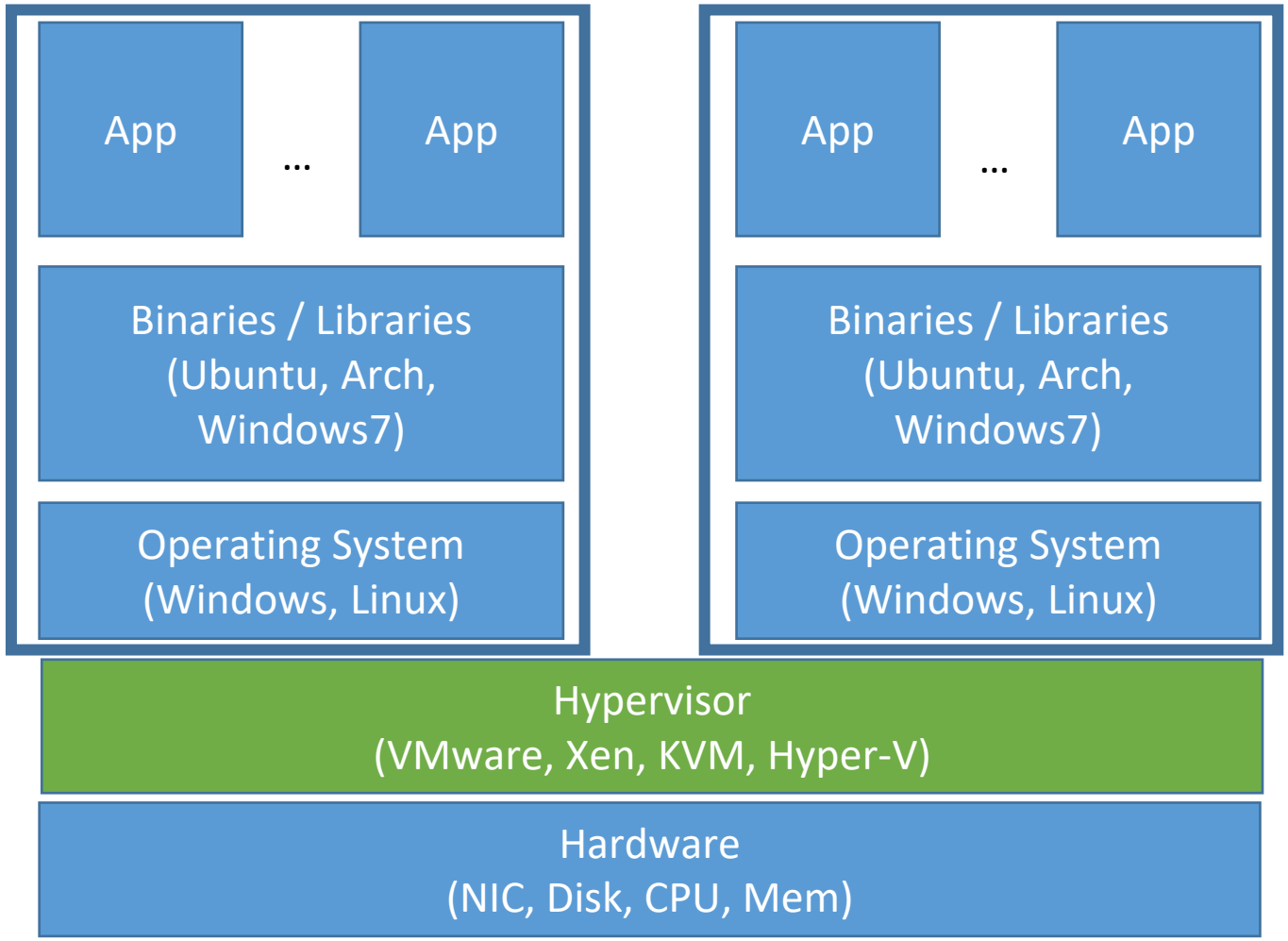
Bare Metal -> Virtual Machines

Run a software layer (hypervisor) that makes it possible to run multiple, independent OSes



Bare Metal -> Virtual Machines

Run a software layer (hypervisor) that makes it possible to run multiple, independent OSes



Challenges (of bare metal):

- Resource wastage (using only part of a server), so want to run multiple applications on the same server

SOLVED (with virtualization):

- Applications might need different Operating Systems
- Applications might have variable workloads
- Applications might have conflicting dependencies (e.g., App1 needs python3, App2 needs python2) - sometimes complex to resolve

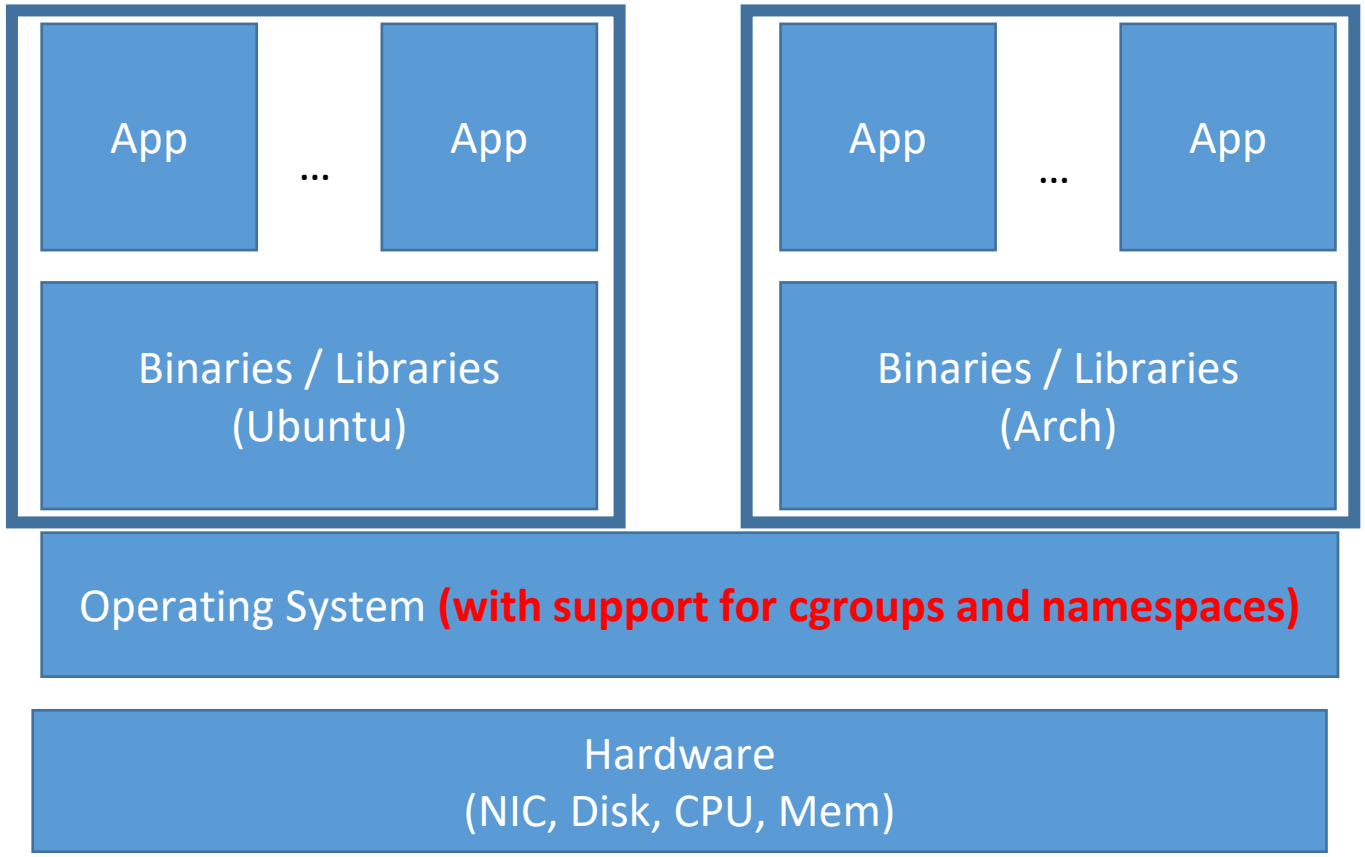
BUT:

- Heavyweight - CPU, Memory, Disk for extra OS and full set of binaries
- Overhead - OS traditionally assume they're directly on hardware. (CPU arch has improved)

What about Containers?

Bare Metal -> Virtual Machines -> Containers

What if we don't need different OSes? Soln: Introduce isolation mechanisms into OS



chroot - 1982

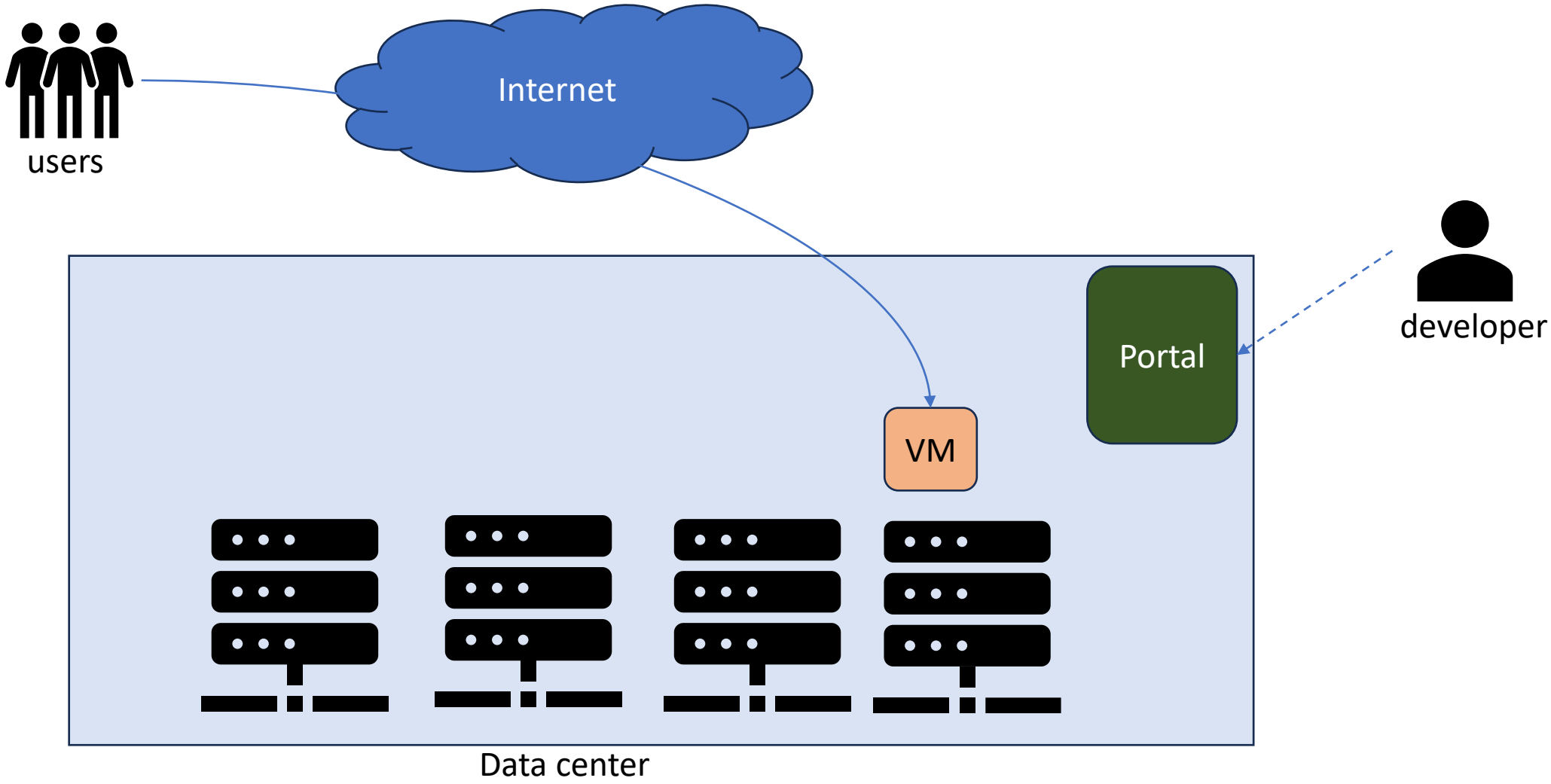
Free BSD Jails - 2000
Virtuozzo OpenVZ - 2000
Linux Vserver - 2001

LXC - 2008 (required no patches to kernel)

namespace - what resources and naming of those resources a process sees (file descriptors, IP addresses)

cgroup - (control group) groups processes and allocates resources (CPU, Memory) that the kernel enforces

Virtual Machines in Cloud Computing





University of Colorado **Boulder**

Datacenter Fabric (VL2)

Course: Networking Principles in Practice – Cloud Networking
Module: Cloud Background



University of Colorado **Boulder**

Microsoft Azure's Network

- SIGCOMM 2009

VL2: A Scalable and Flexible Data Center Network

Albert Greenberg
James R. Hamilton
Navendu Jain
Srikanth Kandula
Changhoon Kim
Parantap Lahiri
David A. Maltz
Parveen Patel
Sudipta Sengupta

Microsoft Research

Abstract

To be agile and cost effective, data centers should allow dynamic resource allocation across large server pools. In particular, the data center network should enable any server to be assigned to any service. To meet these goals, we present VL2, a practical network architecture that scales to support huge data centers with uniform high capacity between servers, performance isolation between services, and Ethernet layer-2 semantics. VL2 uses (1) flat addressing to allow service instances to be placed anywhere in the network, (2) Valiant Load Balancing to spread traffic uniformly across network paths, and (3) end-system based address resolution to scale to large server pools, without introducing complexity to the network control plane. VL2's design is driven by detailed measurements of traffic and fault data from a large operational cloud service provider. VL2's implementation leverages proven network technologies, already available at low cost in high-speed hardware implementations, to build a scalable and reliable network architecture. As a result, VL2 networks can be deployed today, and we have built a working prototype. We evaluate the merits of the VL2 design using measurement, analysis, and experiments. Our VL2 prototype shuffles 2.7 TB of data among 75 servers in 395 seconds – sustaining a rate that is 94% of the maximum possible.

Agility promises improved risk management and cost savings. Without agility, each service must pre-allocate enough servers to meet difficult to predict demand spikes, or risk failure at the brink of success. With agility, the data center operator can meet the fluctuating demands of individual services from a large shared server pool, resulting in higher server utilization and lower costs.

Unfortunately, the designs for today's data center network prevent agility in several ways. First, existing architectures do not provide enough capacity between the servers they interconnect. Conventional architectures rely on tree-like network configurations built from high-cost hardware. Due to the cost of the equipment, the capacity between different branches of the tree is typically oversubscribed by factors of 1:5 or more, with paths through the highest levels of the tree oversubscribed by factors of 1:80 to 1:240. This limits communication between servers to the point that it fragments the server pool — congestion and computation hot-spots are prevalent even when spare capacity is available elsewhere. Second, while data centers host multiple services, the network does little to prevent a traffic flood in one service from affecting the other services around it — when one service experiences a traffic flood, it is common for all those sharing the same network sub-tree to suffer collateral damage. Third, the routing design in conventional networks achieves scale by

(Recall) Hyper scale: Costs

- Costs of electricity, network, etc. (economies of scale)

Table 2: Economies of scale in 2006 for medium-sized datacenter (≈1000 servers) vs. very large datacenter (≈50,000 servers). [24]

Technology	Cost in Medium-sized DC	Cost in Very Large DC	Ratio
Network	$95 per Mbit/sec/month	$13 per Mbit/sec/month	7.1
Storage	$2.20 per GByte / month	$0.40 per GByte / month	5.7
Administration	≈140 Servers / Administrator	>1000 Servers / Administrator	7.1

https://www2.eecs.berkeley.edu/Pubs/TechRpts/2009/EECS-2009-28.pdf

The other side of cost

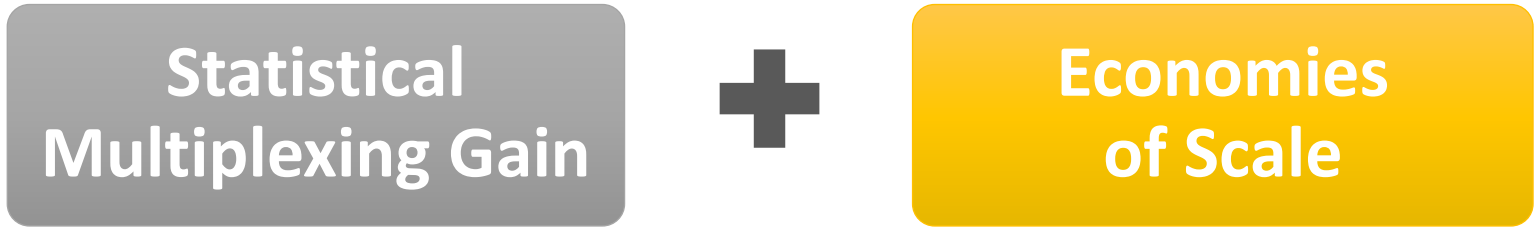
- From VL2 paper: “The cost is also large – upwards of $12 million per month for a 100,000 server data center” (in 2009)
- “AWS spending 12B on 5 new datacenters”

https://www.techradar.com/news/aws-is-spending-billions-on-five-new-data-centers



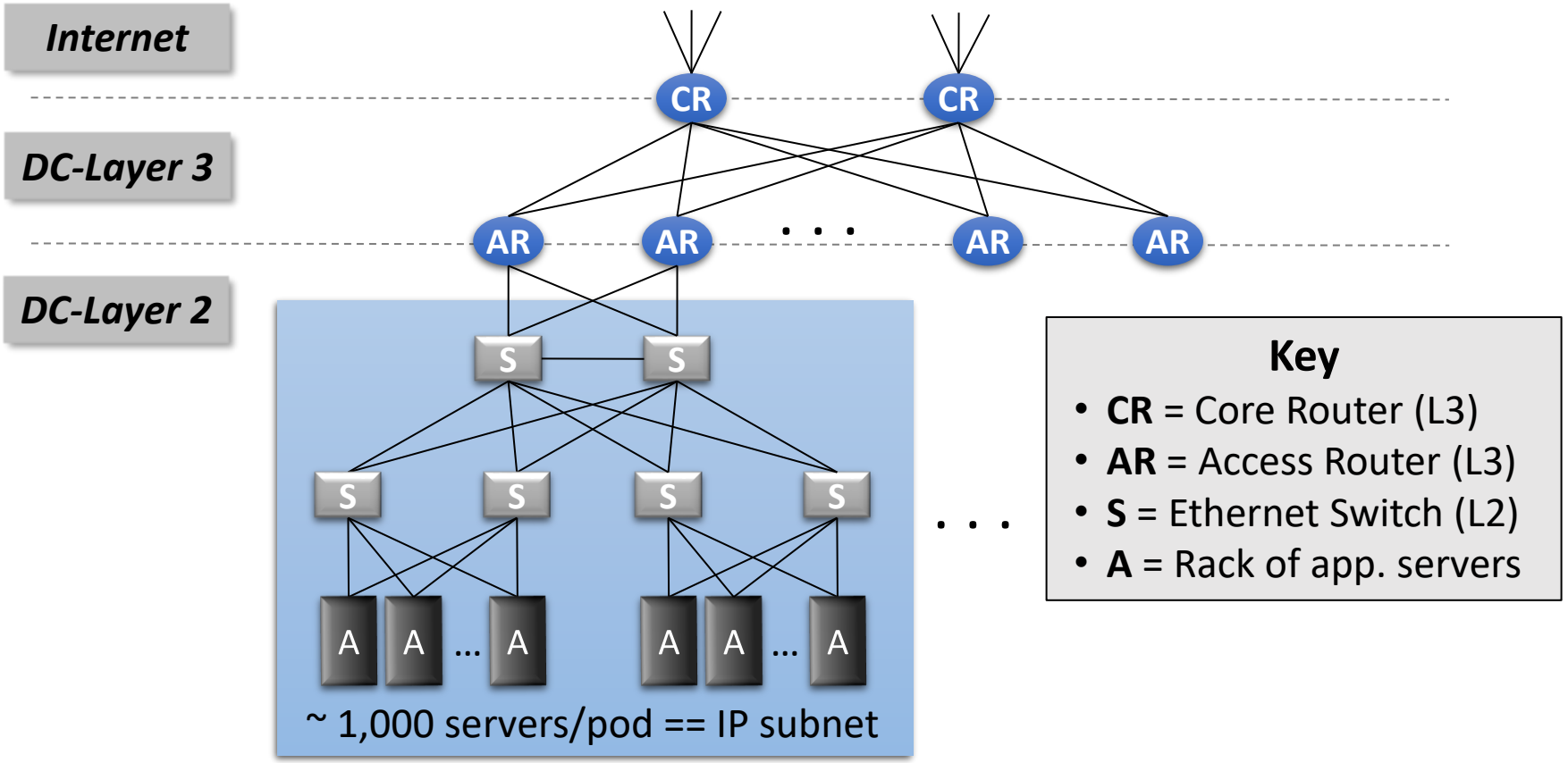
Tenets of a Cloud Scale Data Center

- **Agility**: Assign any servers to any services
  - Boosts cloud utilization
- **Scaling out**: Use large pools of commodities
  - Achieves reliability, performance, low cost



Slides: Changhoon Kim. https://people.eecs.berkeley.edu/~randy/Courses/CS268.F09/lectures/08b-VL2a.pptx

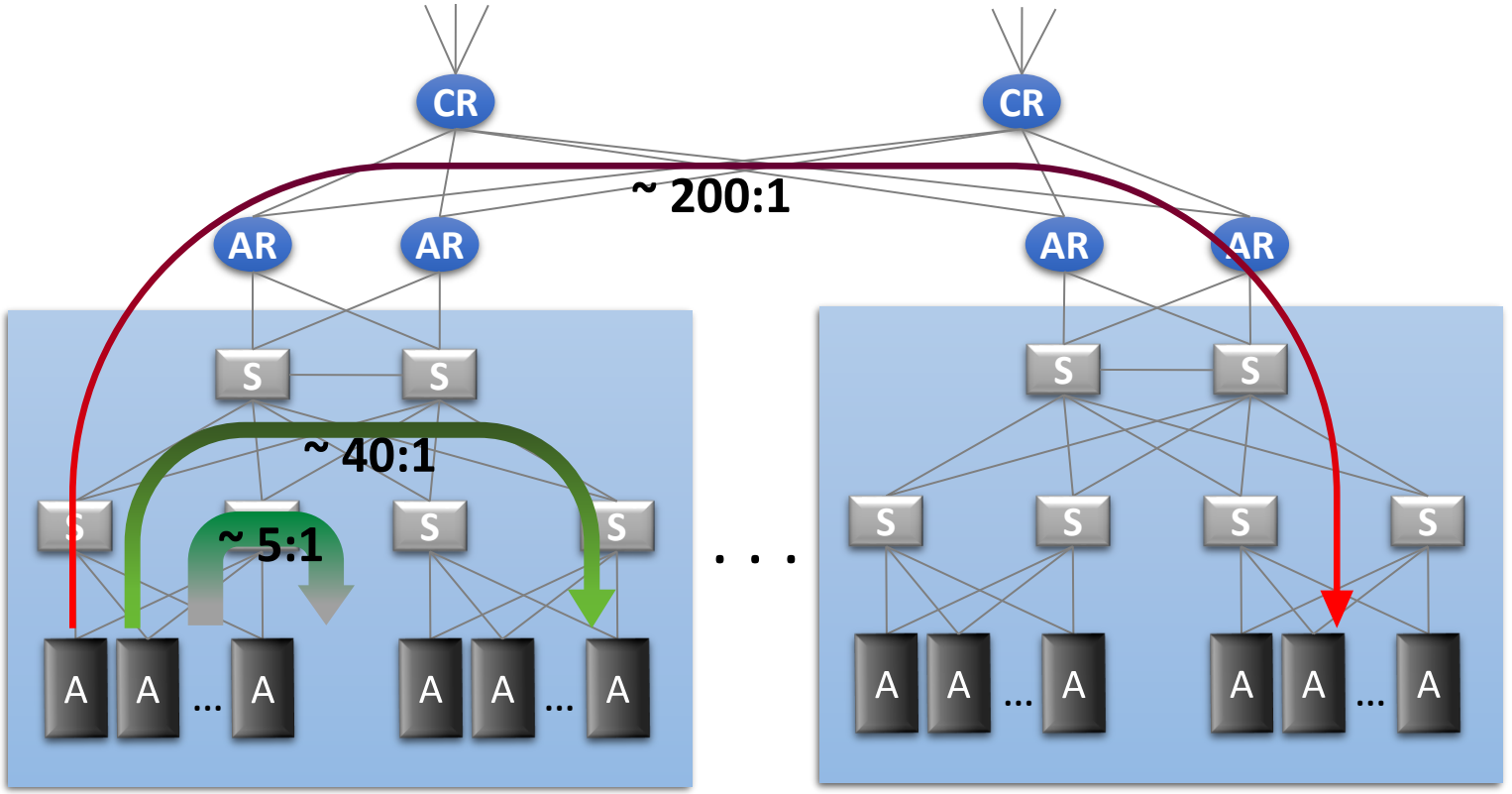
Status Quo: Conventional DC Network



Reference – "Data Center: Load balancing Data Center Services", Cisco 2004

Slides: Changhoon Kim. https://people.eecs.berkeley.edu/~randy/Courses/CS268.F09/lectures/08b-VL2a.pptx

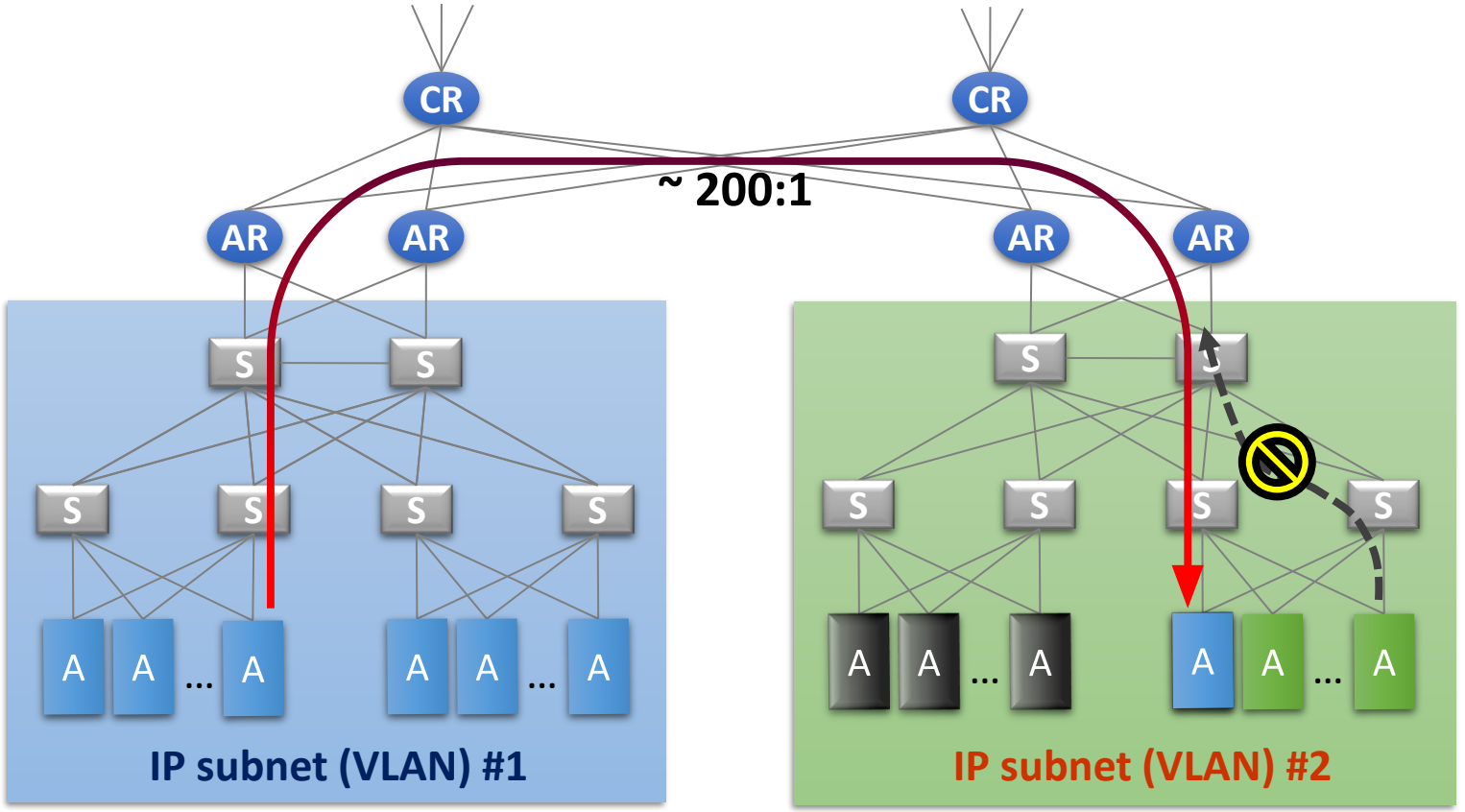
Conventional DC Network Problems



- Dependence on high-cost proprietary routers
- Extremely limited server-to-server capacity

Slides: Changhoon Kim. https://people.eecs.berkeley.edu/~randy/Courses/CS268.F09/lectures/08b-VL2a.pptx

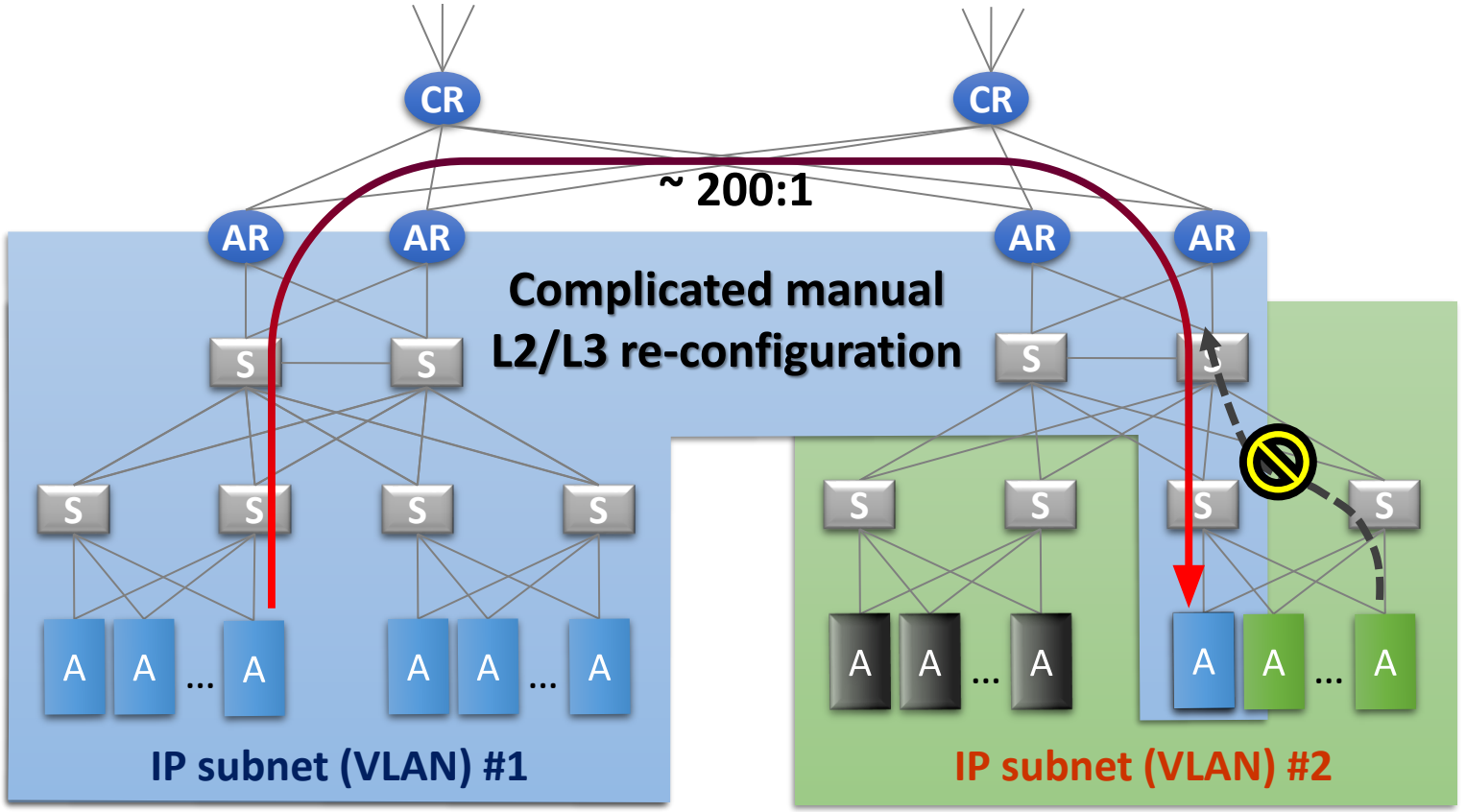
And More Problems …



- Resource fragmentation, significantly lowering cloud utilization (and cost-efficiency)

Slides: Changhoon Kim. https://people.eecs.berkeley.edu/~randy/Courses/CS268.F09/lectures/08b-VL2a.pptx

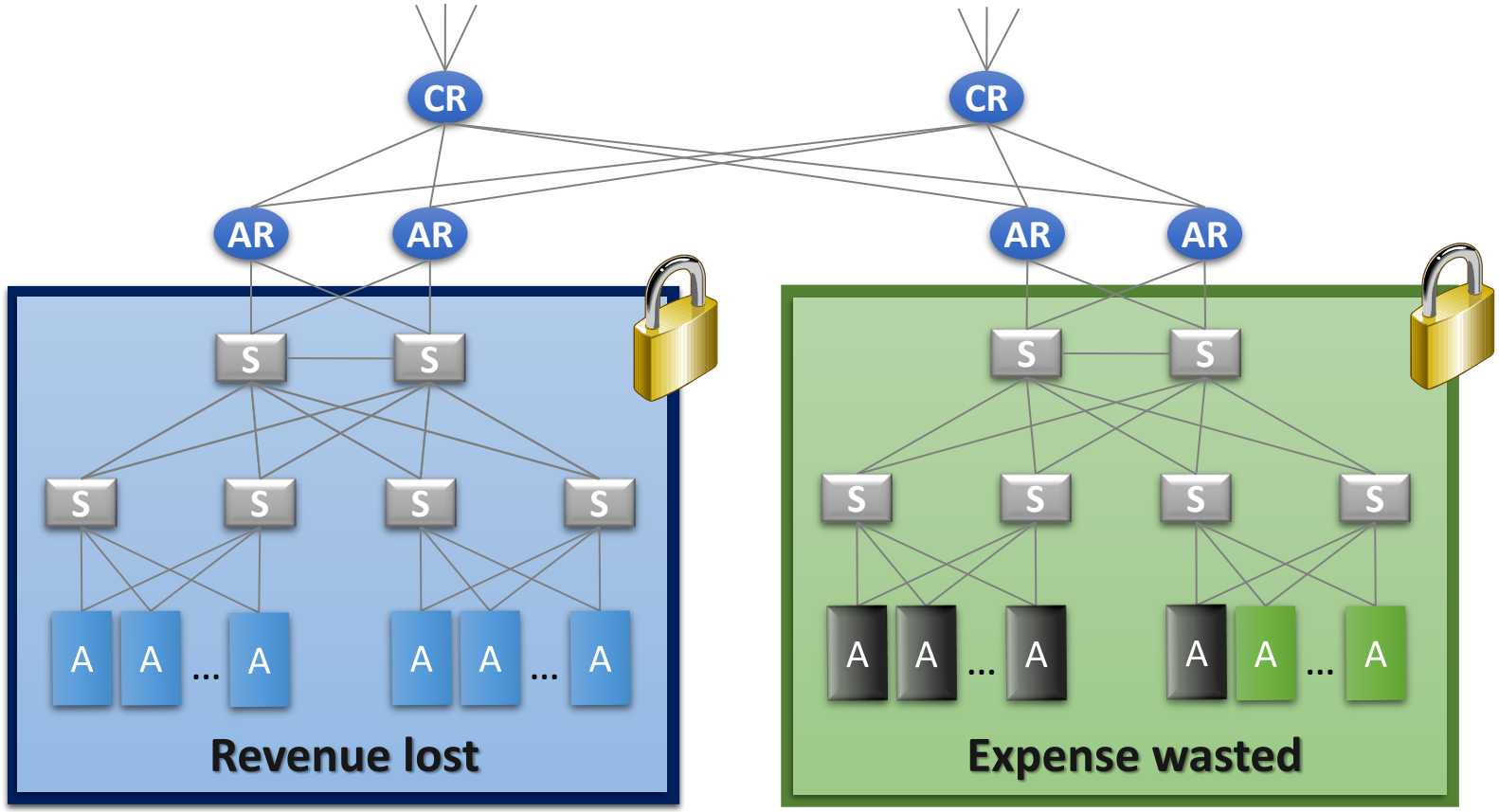
And More Problems …



- Resource fragmentation, significantly lowering cloud utilization (and cost-efficiency)

Slides: Changhoon Kim. https://people.eecs.berkeley.edu/~randy/Courses/CS268.F09/lectures/08b-VL2a.pptx

And More Problems …



- Resource fragmentation, significantly lowering cloud utilization (and cost-efficiency)

Slides: Changhoon Kim. https://people.eecs.berkeley.edu/~randy/Courses/CS268.F09/lectures/08b-VL2a.pptx

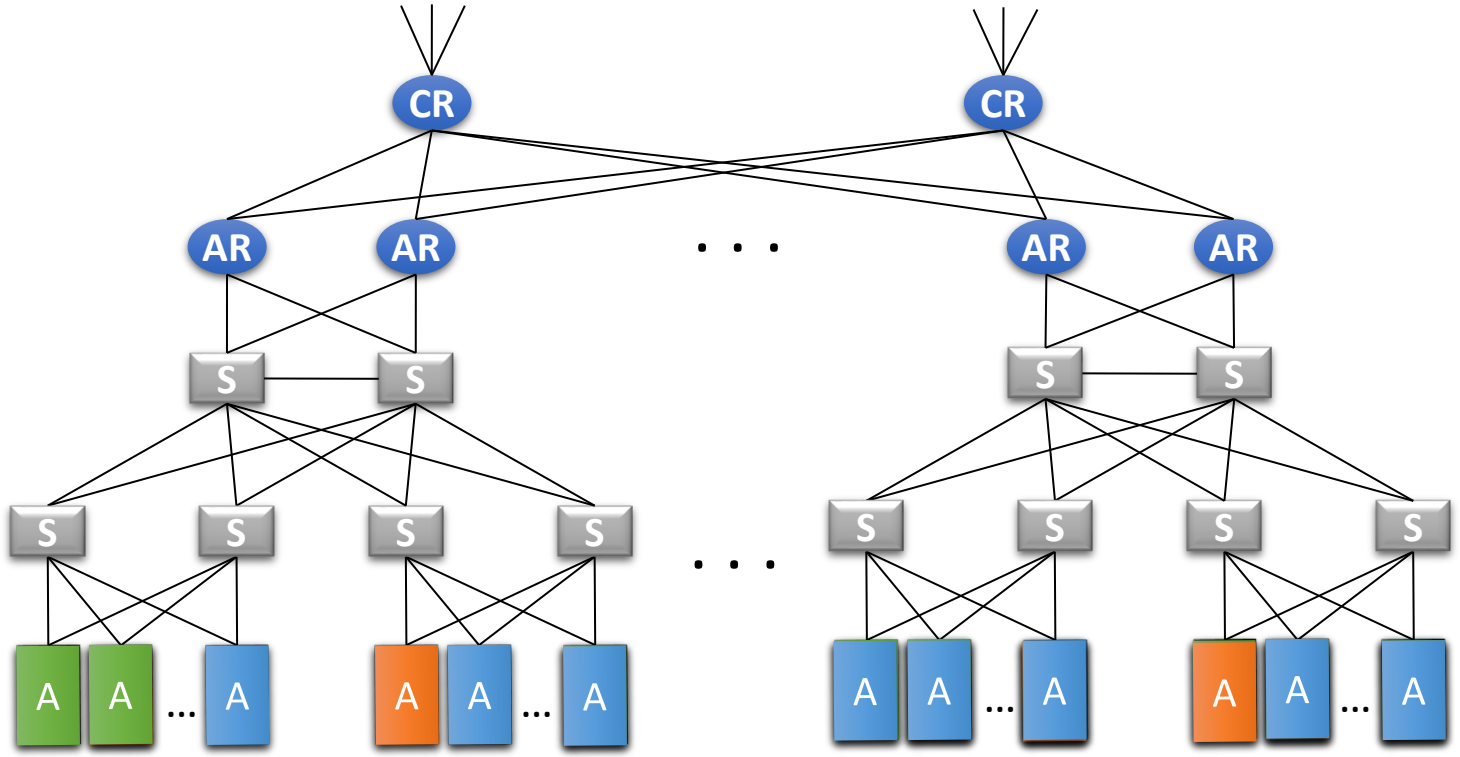
Know Your Cloud DC: Challenges

- Instrumented a large cluster used for data mining and identified distinctive traffic patterns
- Traffic patterns are **highly volatile**
  - A large number of distinctive patterns even in a day
- Traffic patterns are **unpredictable**
  - Correlation between patterns very weak

Optimization should be done frequently and rapidly

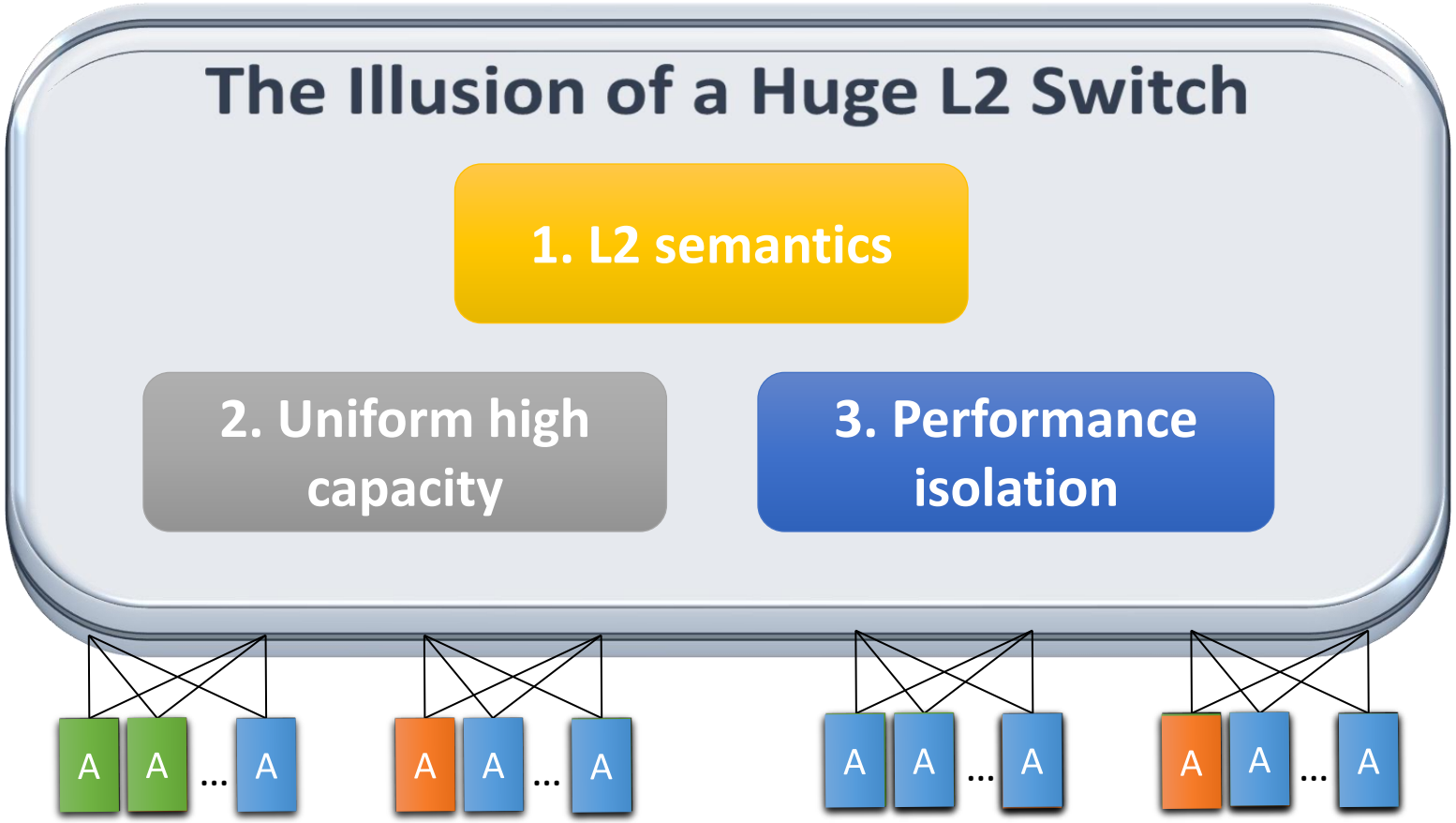
Slides: Changhoon Kim. https://people.eecs.berkeley.edu/~randy/Courses/CS268.F09/lectures/08b-VL2a.pptx

All We Need is Just a Huge L2 Switch, or an Abstraction of One



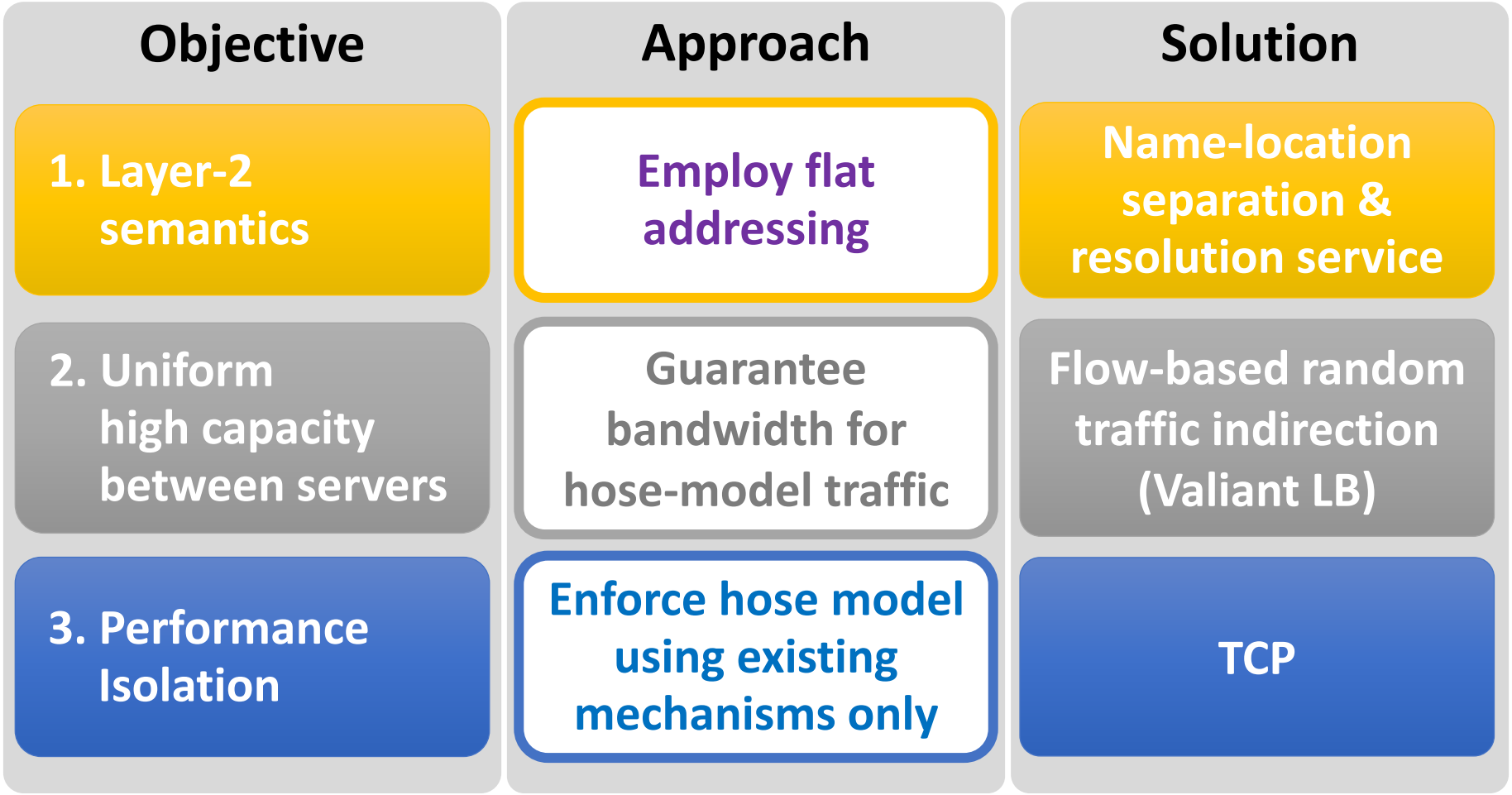
Slides: Changhoon Kim. https://people.eecs.berkeley.edu/~randy/Courses/CS268.F09/lectures/08b-VL2a.pptx

All We Need is Just a Huge L2 Switch, or an Abstraction of One



Slides: Changhoon Kim. https://people.eecs.berkeley.edu/~randy/Courses/CS268.F09/lectures/08b-VL2a.pptx

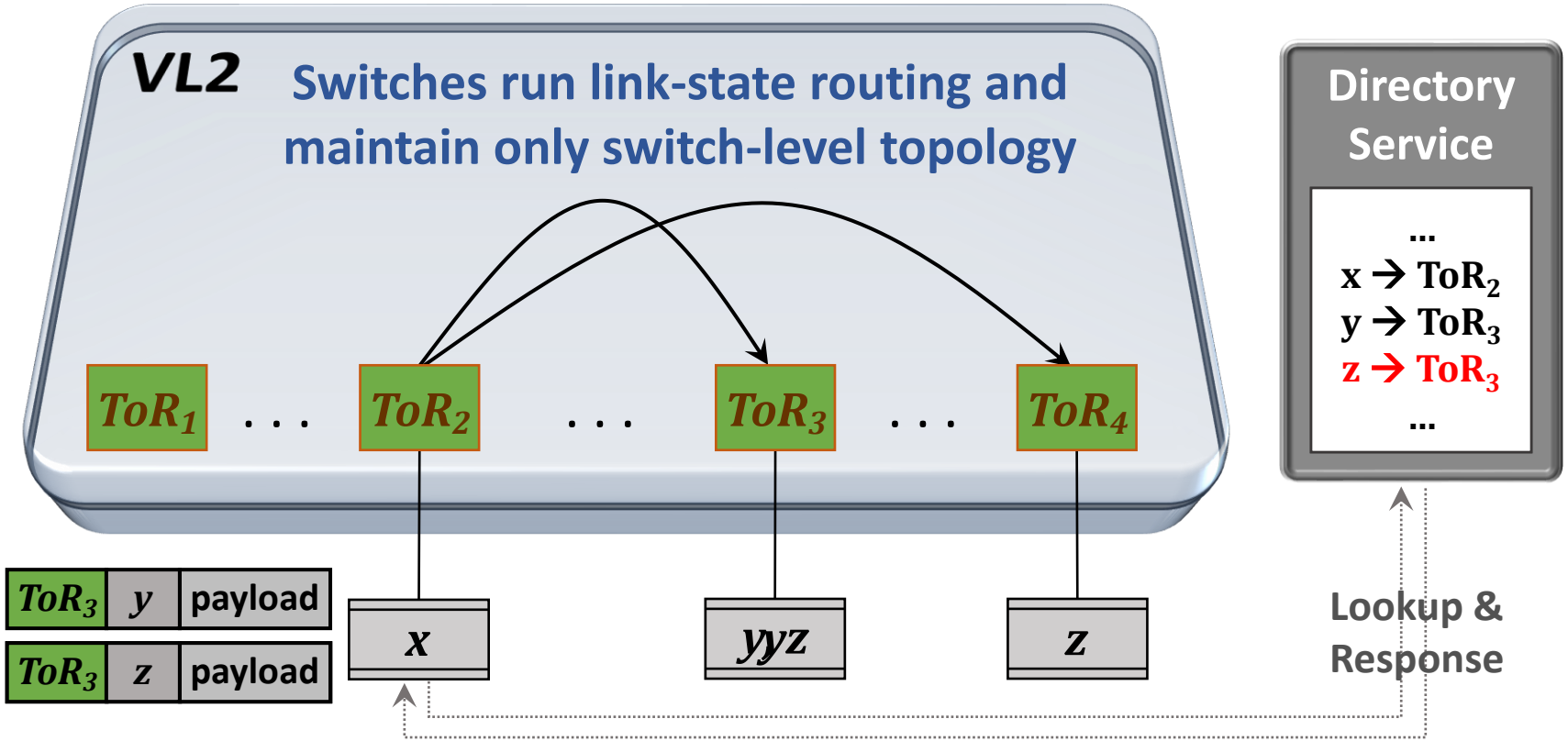
Specific Objectives and Solutions



Slides: Changhoon Kim. https://people.eecs.berkeley.edu/~randy/Courses/CS268.F09/lectures/08b-VL2a.pptx

Addressing and Routing: Name-Location Separation

Cope with host churns with very little overhead

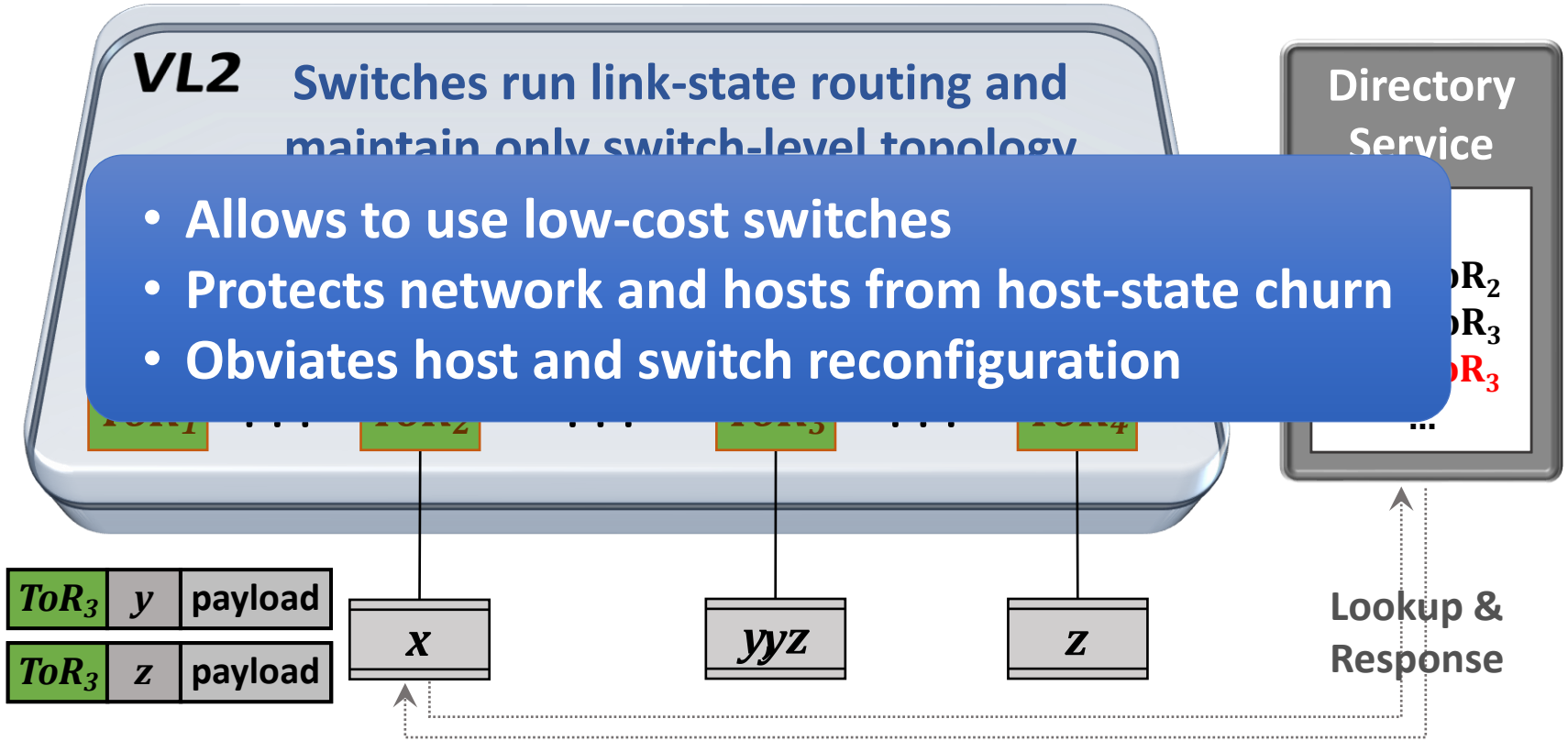


Servers use flat names

Slides: Changhoon Kim. https://people.eecs.berkeley.edu/~randy/Courses/CS268.F09/lectures/08b-VL2a.pptx

Addressing and Routing: Name-Location Separation

Cope with host churns with very little overhead

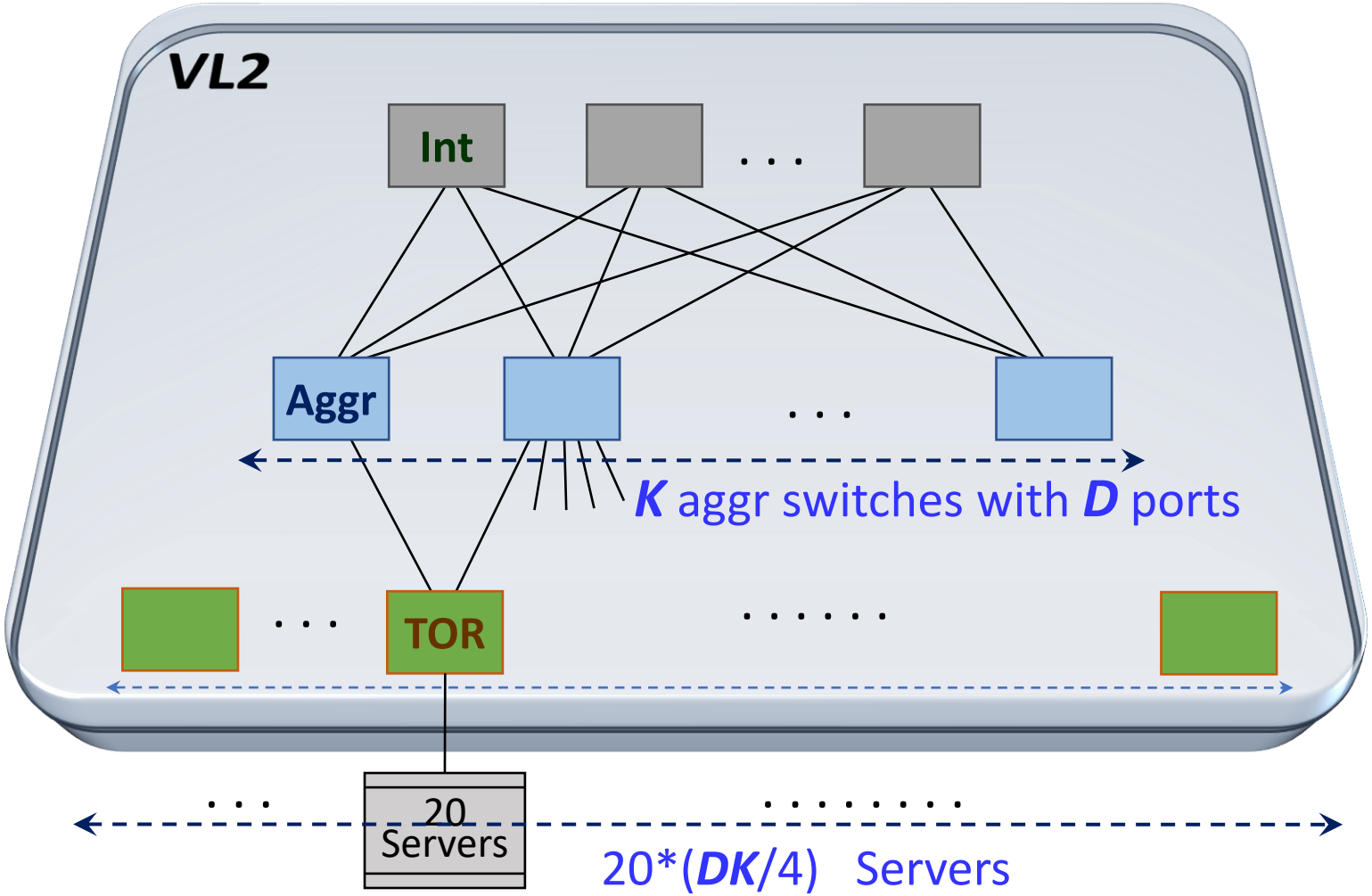


Servers use flat names

Slides: Changhoon Kim. https://people.eecs.berkeley.edu/~randy/Courses/CS268.F09/lectures/08b-VL2a.pptx

Example Topology: Clos Network

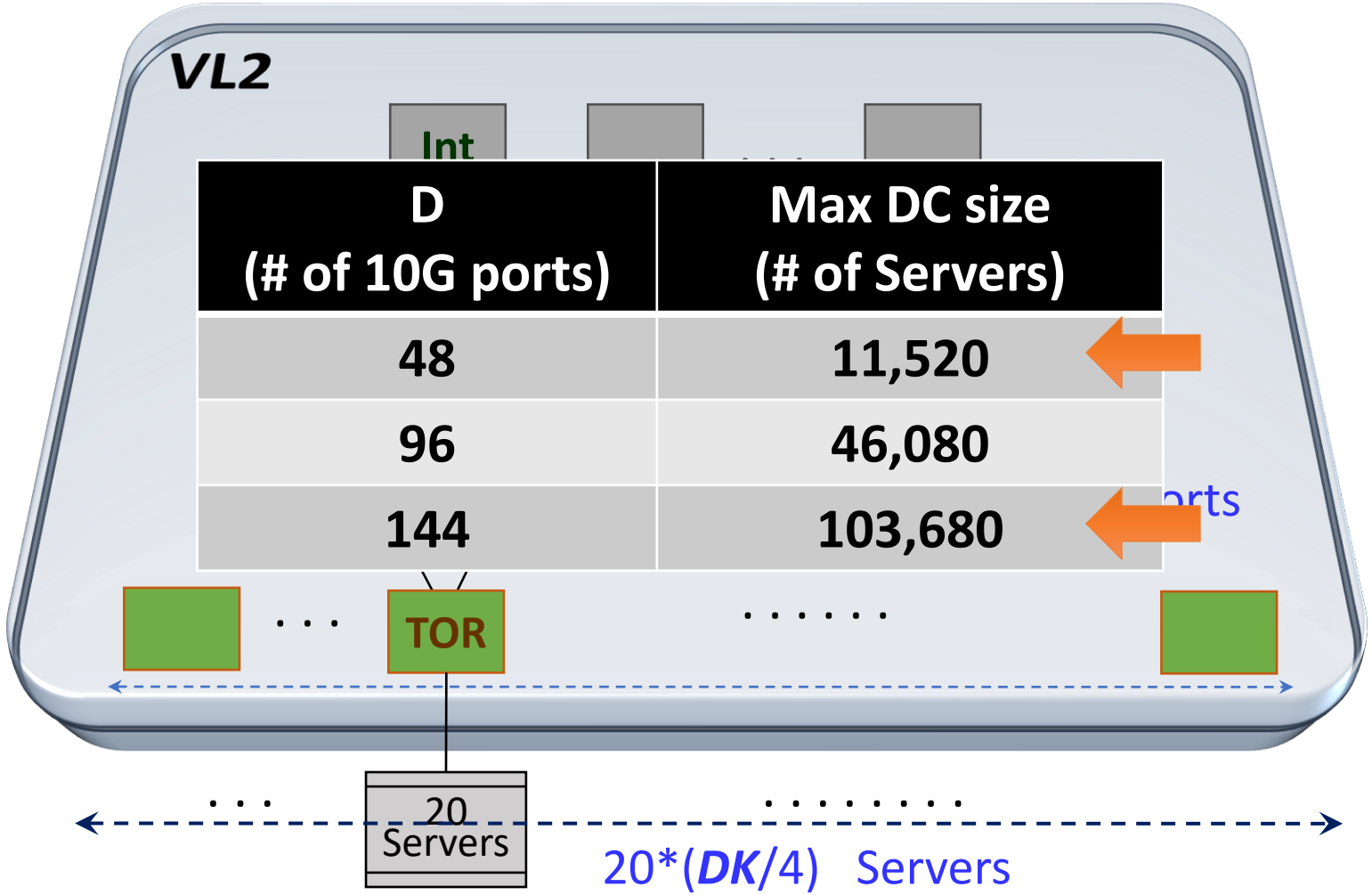
Offer huge aggr capacity and multi paths at modest cost



Slides: Changhoon Kim. https://people.eecs.berkeley.edu/~randy/Courses/CS268.F09/lectures/08b-VL2a.pptx

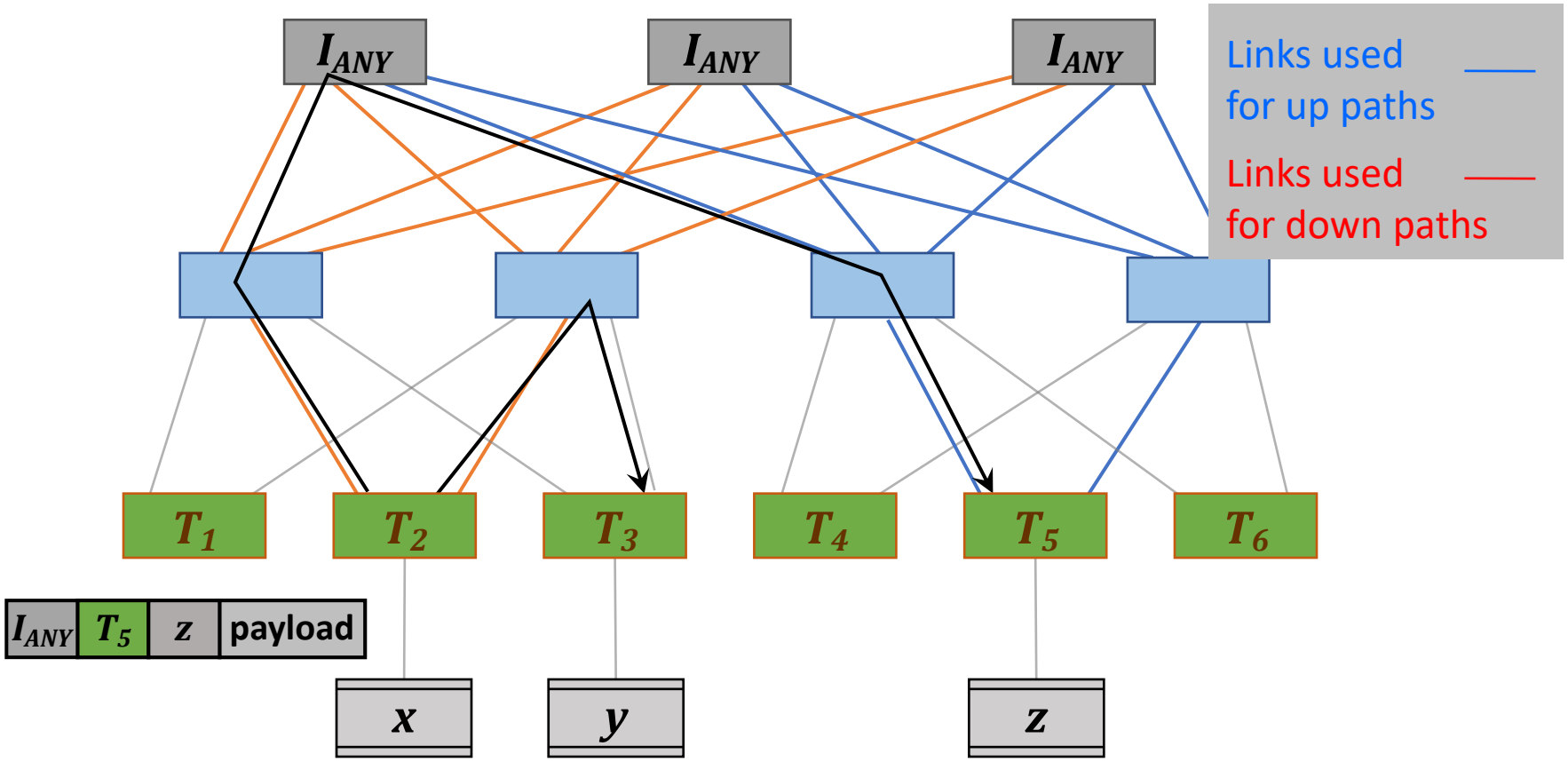
Example Topology: Clos Network

Offer huge aggr capacity and multi paths at modest cost



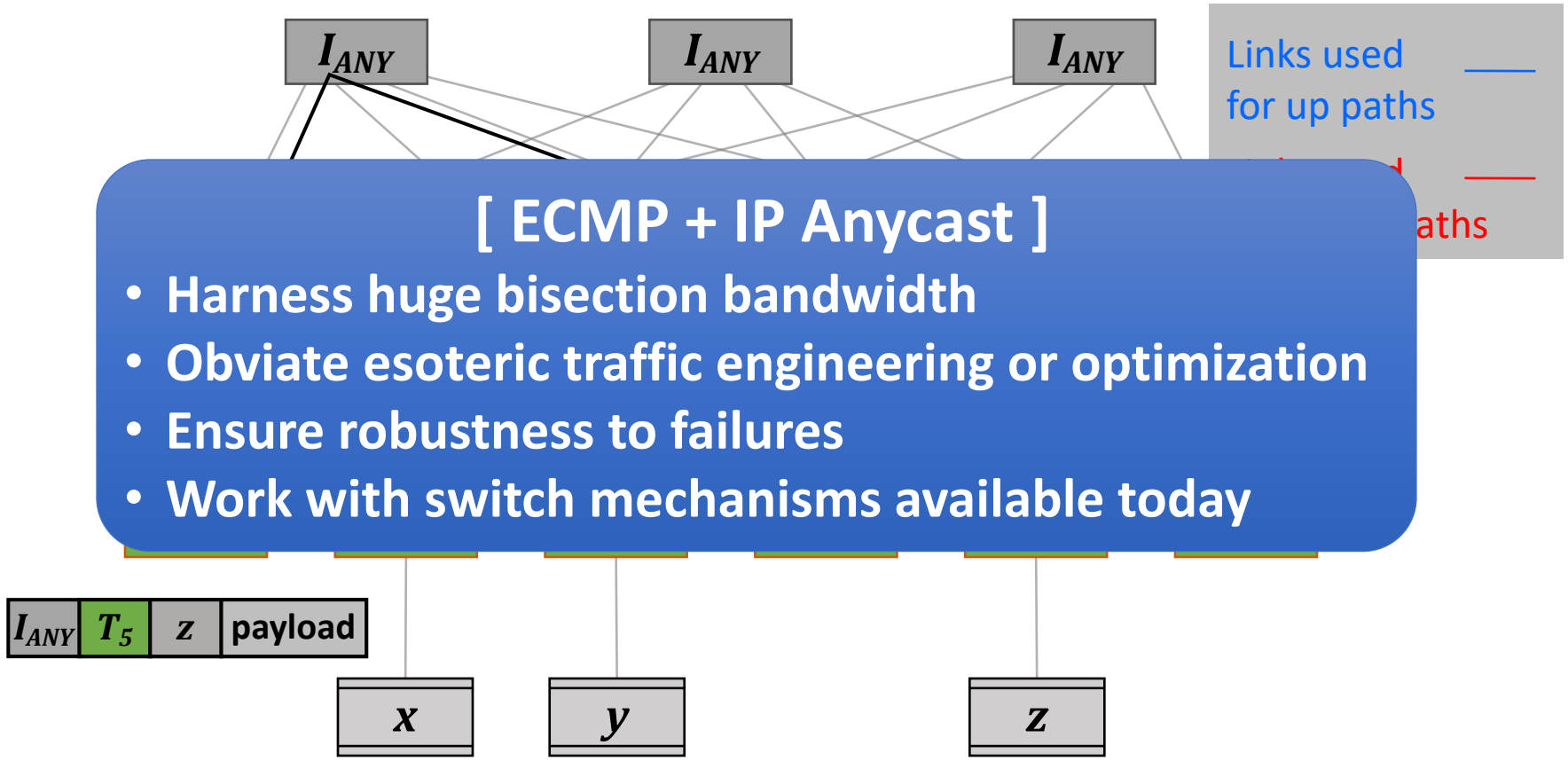
Traffic Forwarding: Random Indirection

Cope with arbitrary TMs with very little overhead



Traffic Forwarding: Random Indirection

Cope with arbitrary TMs with very little overhead



VL2 Conclusion

- VL2 achieves **agility at scale** via
  1. L2 semantics
  2. Uniform high capacity between servers
  3. Performance isolation between services

Lessons

- **Randomization can tame volatility**
- **Add functionality where you have control**
- **There's no need to wait!**



University of Colorado **Boulder**

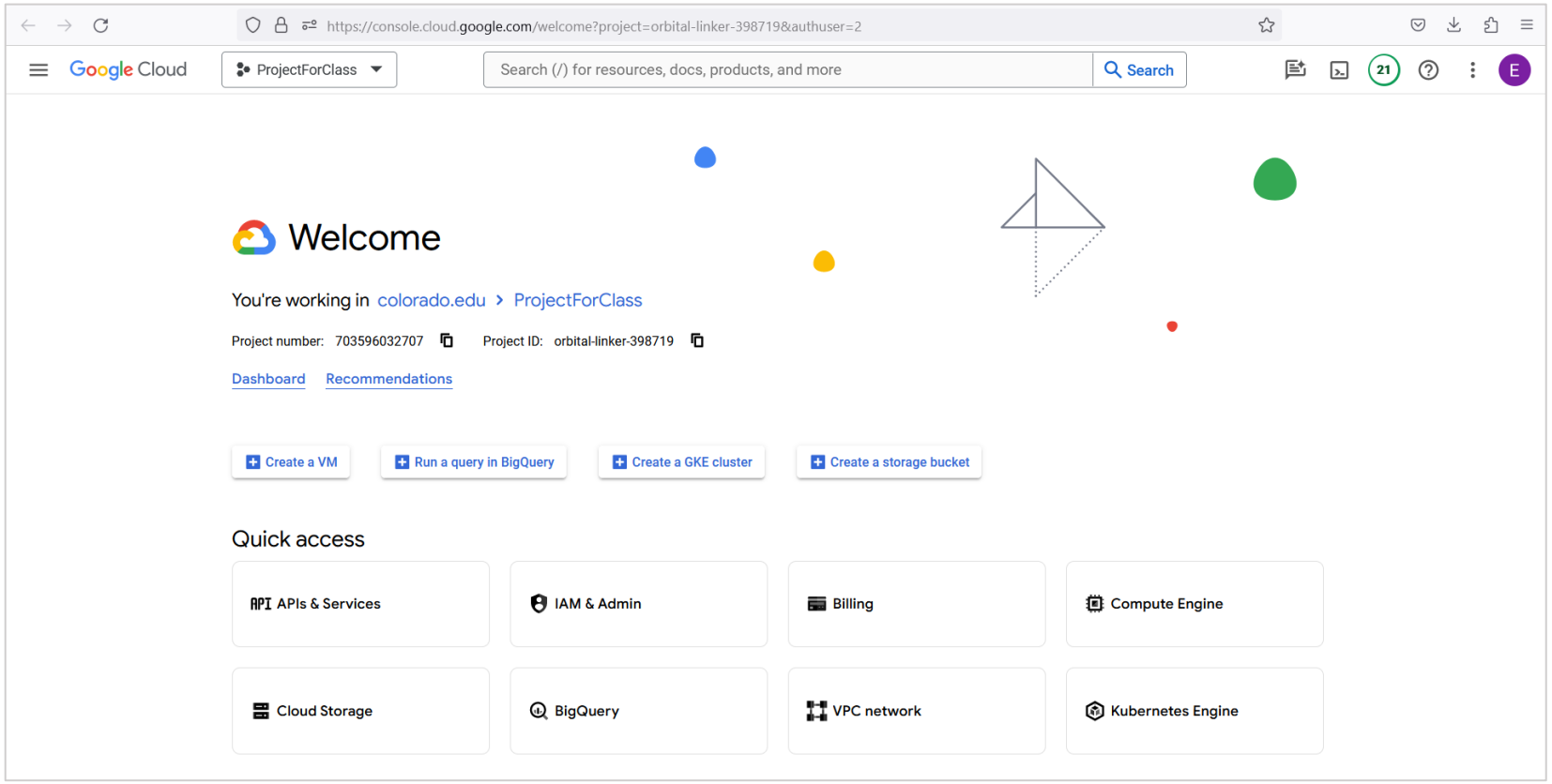
Google Cloud Platform

Course: Networking Principles in Practice – Cloud Networking
Module: Cloud Background



University of Colorado **Boulder**

Google Cloud Platform (GCP)



Google Cloud Platform

Note: $300 credit for new users

Walkthrough Demo

- Create a VM
- ping it (incl. how to find it’s IP address)
- gcloud init
- ssh into it (using gcloud, and how to get the gcloud command)



University of Colorado **Boulder**

Terraform Overview

Course: Networking Principles in Practice – Cloud Networking
Module: Cloud Background



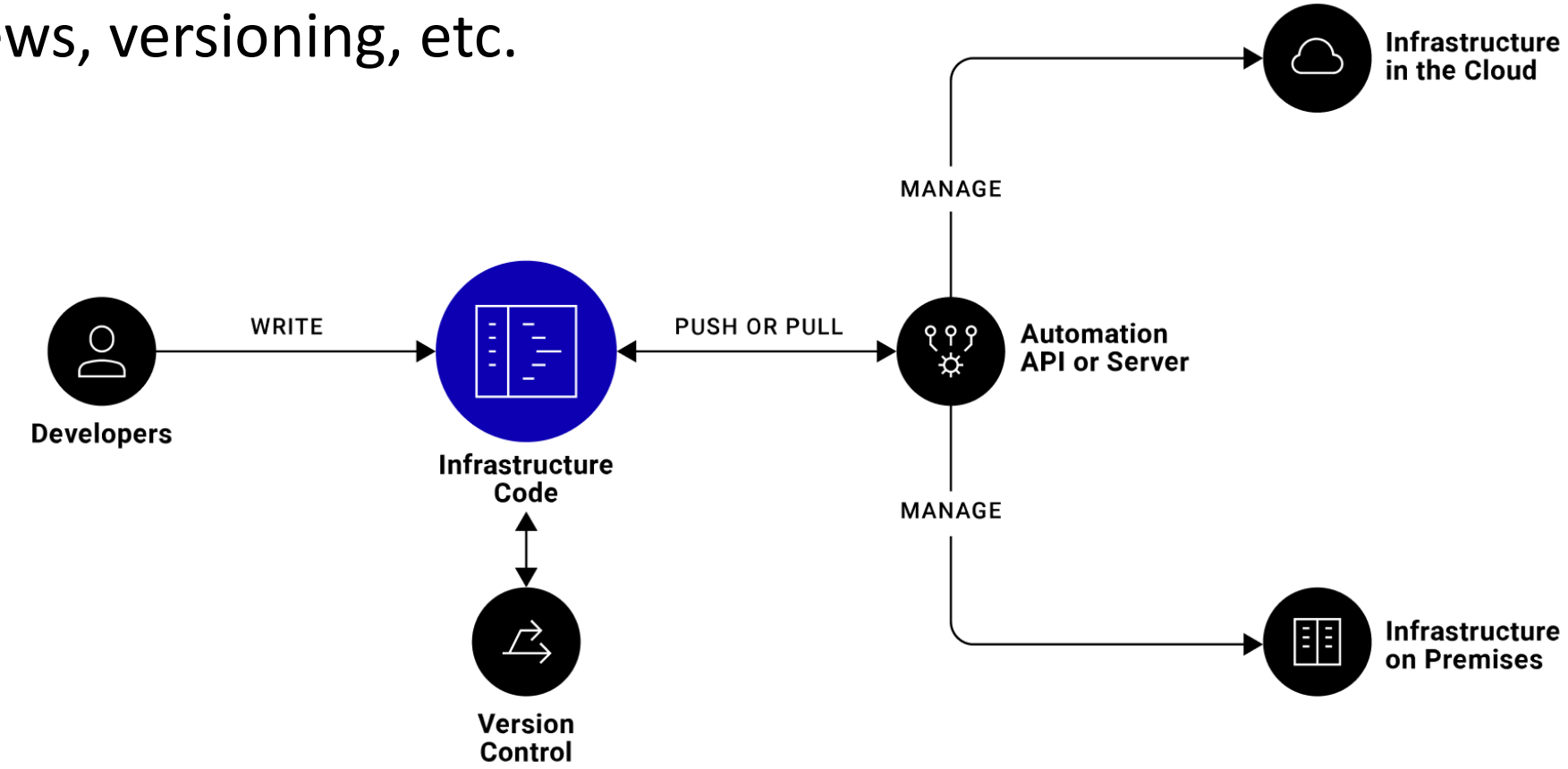
University of Colorado **Boulder**

Install terraform, gcloud, and vscode

- https://developer.hashicorp.com/terraform/install
- https://cloud.google.com/sdk/docs/install
- https://code.visualstudio.com/

Infrastructure as Code

- Software practices applied to infrastructure.
- Code reviews, versioning, etc.



Infrastructure as Code Use

Provisioning Infrastructure

- Launching VMs
- Specifying the network
- …

Deploying Application

- Configuring OS
- Configuring packages
- …

Infrastructure as Code Use

Provisioning Infrastructure

- Launching VMs
- Specifying the network



Deploying Application

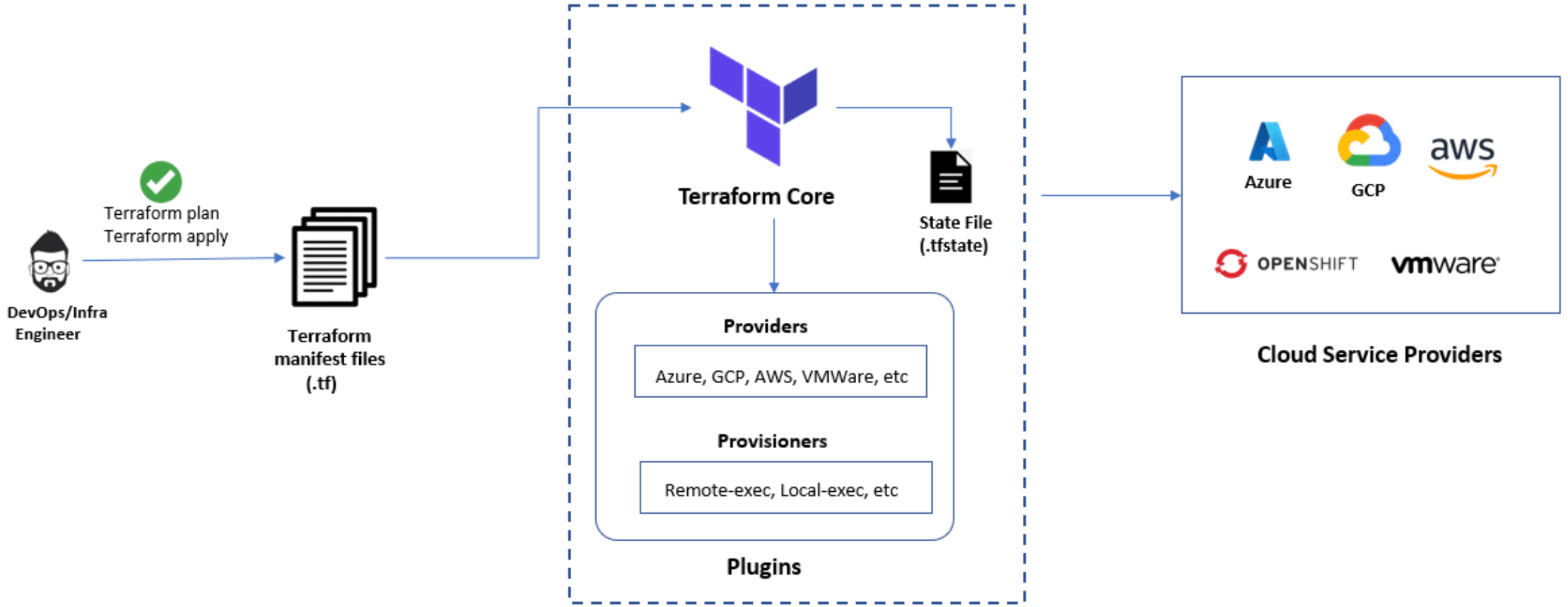
- Configuring OS
- Configuring packages



IaC Provisioning Tools

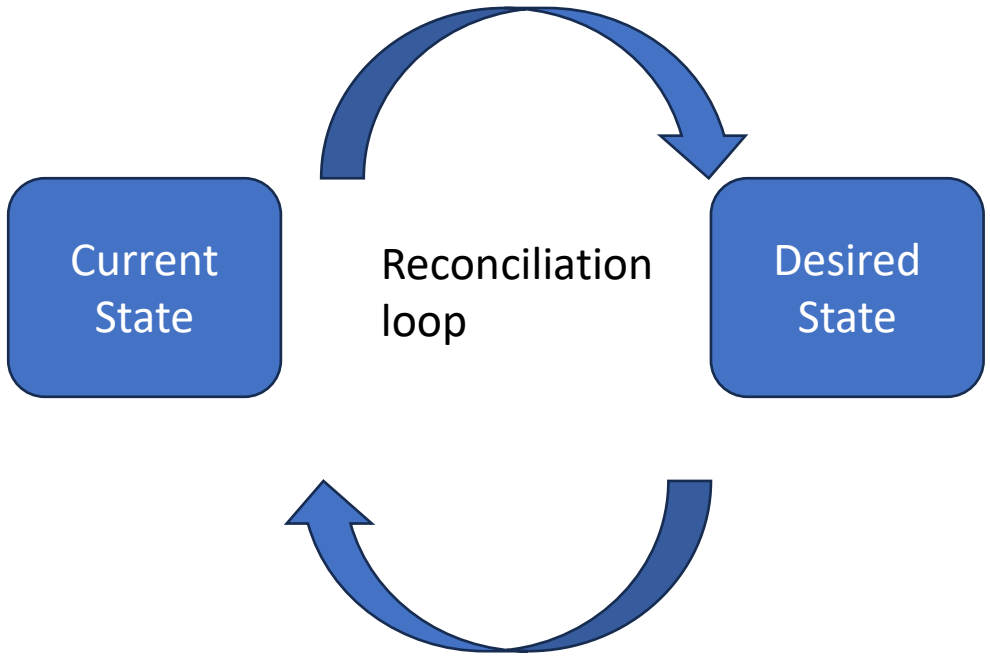
- Cloud Specific
  - AWS Cloud Formation, Google Deployment Manager, Azure Resource Manager
- Agnostic
  - Terraform, Pulumi

Terraform Architecture



Declarative vs Imperative

- Imperative – when you say how to get what you want
- Declarative - when you say what you want



Terraform

Providers

- https://registry.terraform.io/browse/providers

Syntax

Arguments

- Assigns value to a name

```
image_id = "abc123"
```

Block

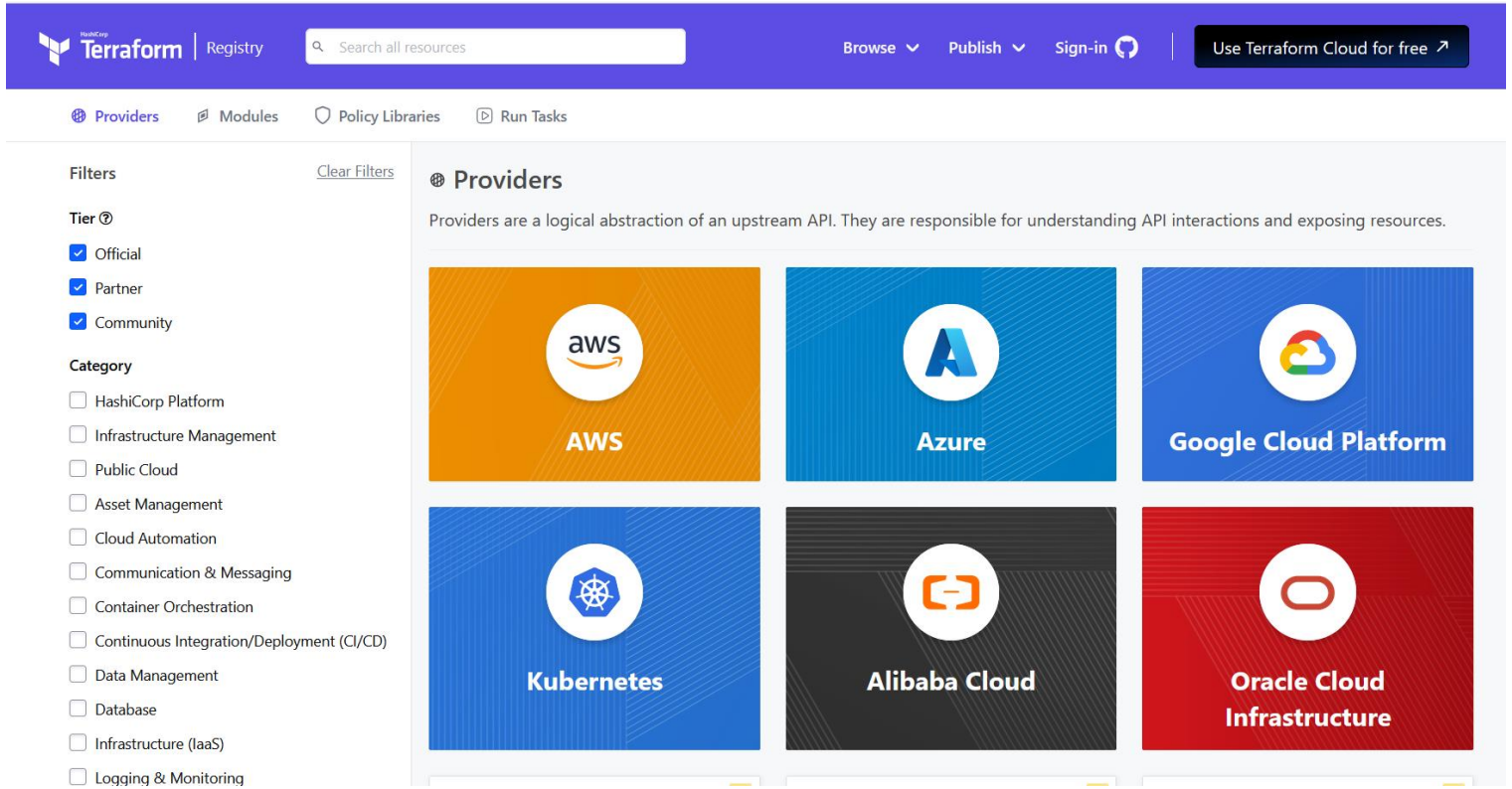
- Container for other content
- Has type (resource, provider, etc.) and 0 or more labels (determined by type)
- Can be nested

```
resource "aws_instance" "example" {
  ami = "abc123"

  network_interface {
    # ...
  }
}
```

Provider Block

- https://registry.terraform.io/browse/providers

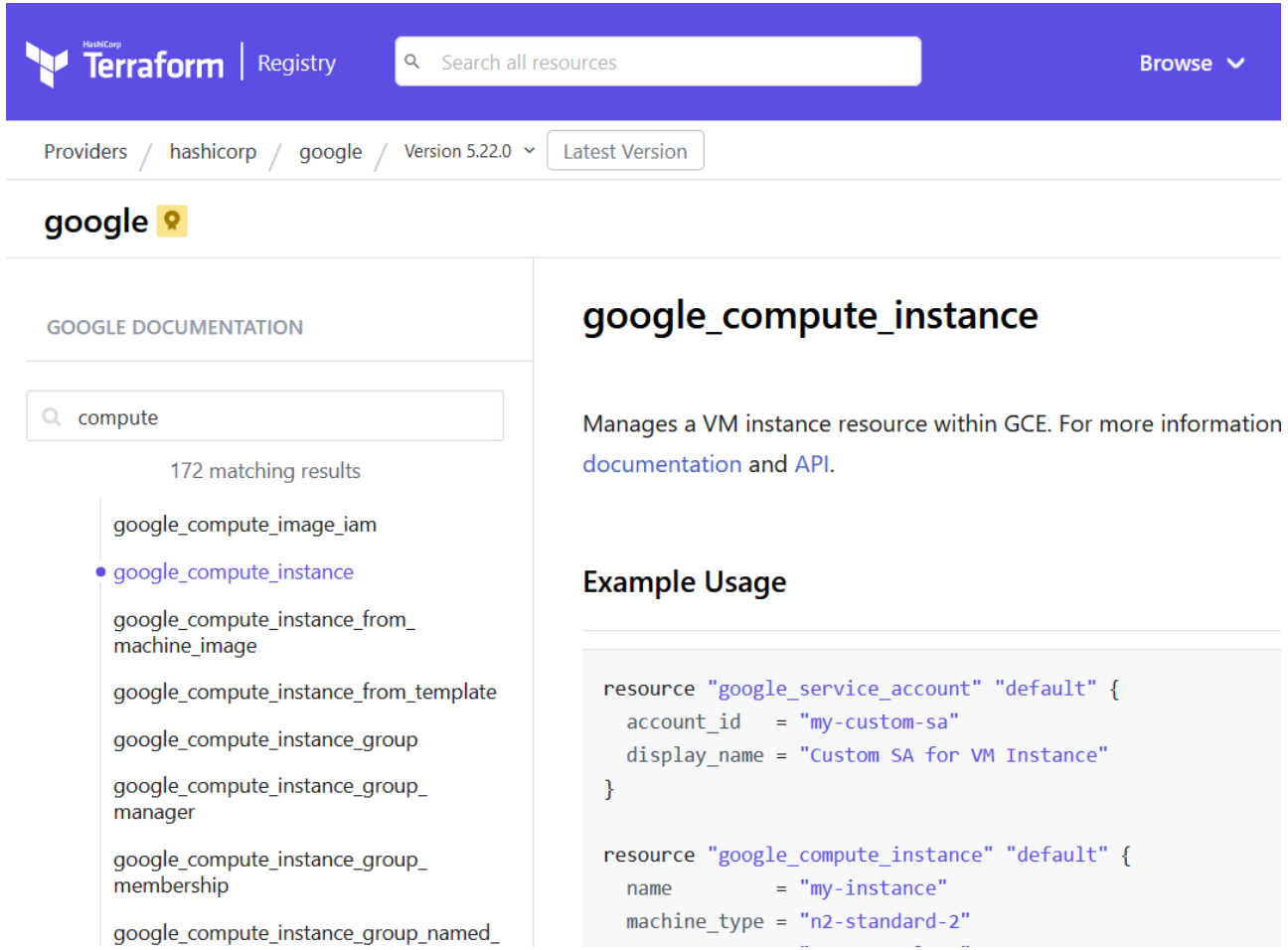



```
provider "google" {
  project = "acme-app"
  region  = "us-central1"
}
```

Resource Block

```
resource "aws_instance" "example" {
  ami = "abc123"

  network_interface {
    # ...
  }
}
```



https://registry.terraform.io/providers/hashicorp/google/latest/docs/resources/compute_instance

Other Features

- variables,
- conditions,
- loop like construct,
- dependency specification,
- functions,

Really nice video on Youtube:
https://www.youtube.com/watch?v=7xngnjfIlK4&ab_channel=DevOpsDirective

Summary

- Infrastructure as Code
- Provisioning Infrastructure
- Terraform
- Google Compute Platform as a provider



University of Colorado **Boulder**

Terraform With GCP

Course: Networking Principles in Practice – Cloud Networking
Module: Cloud Background



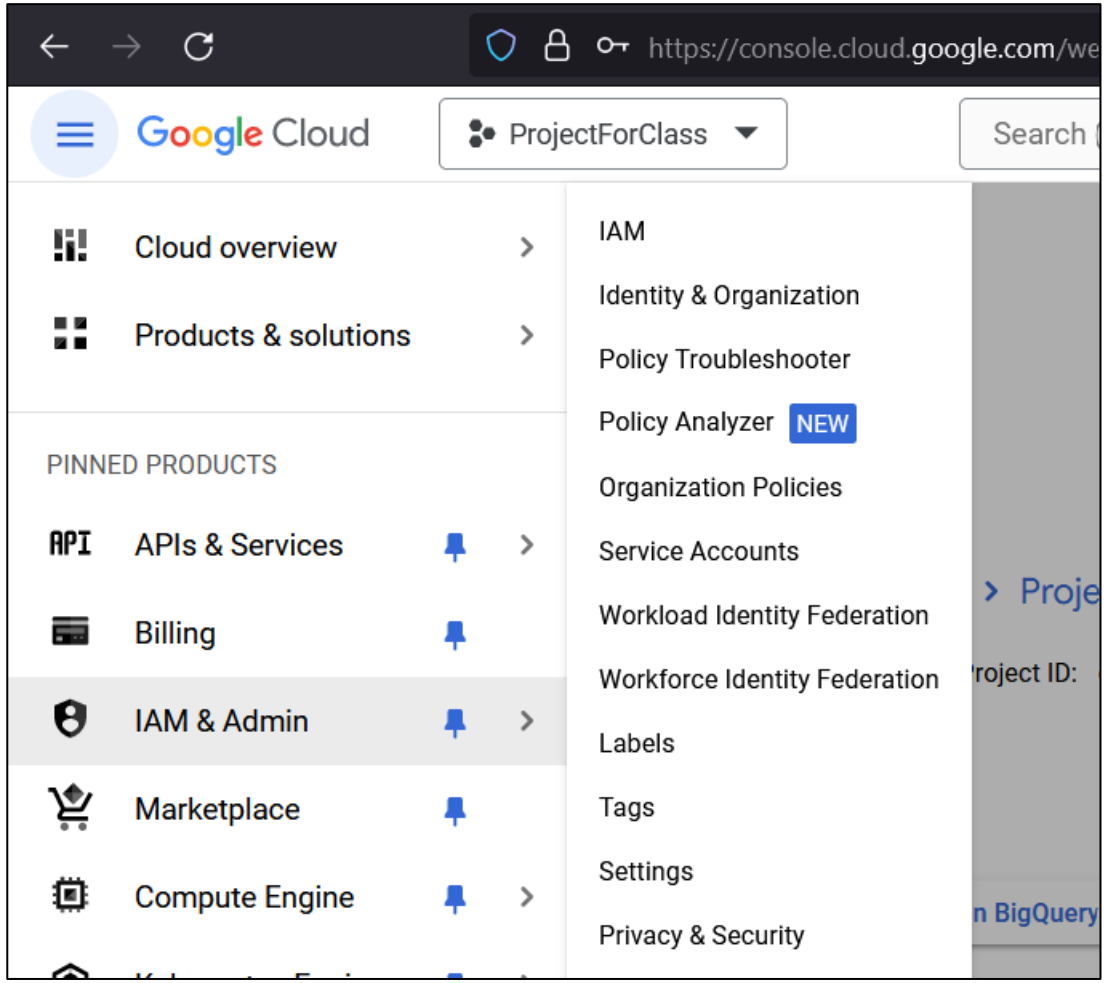
University of Colorado **Boulder**

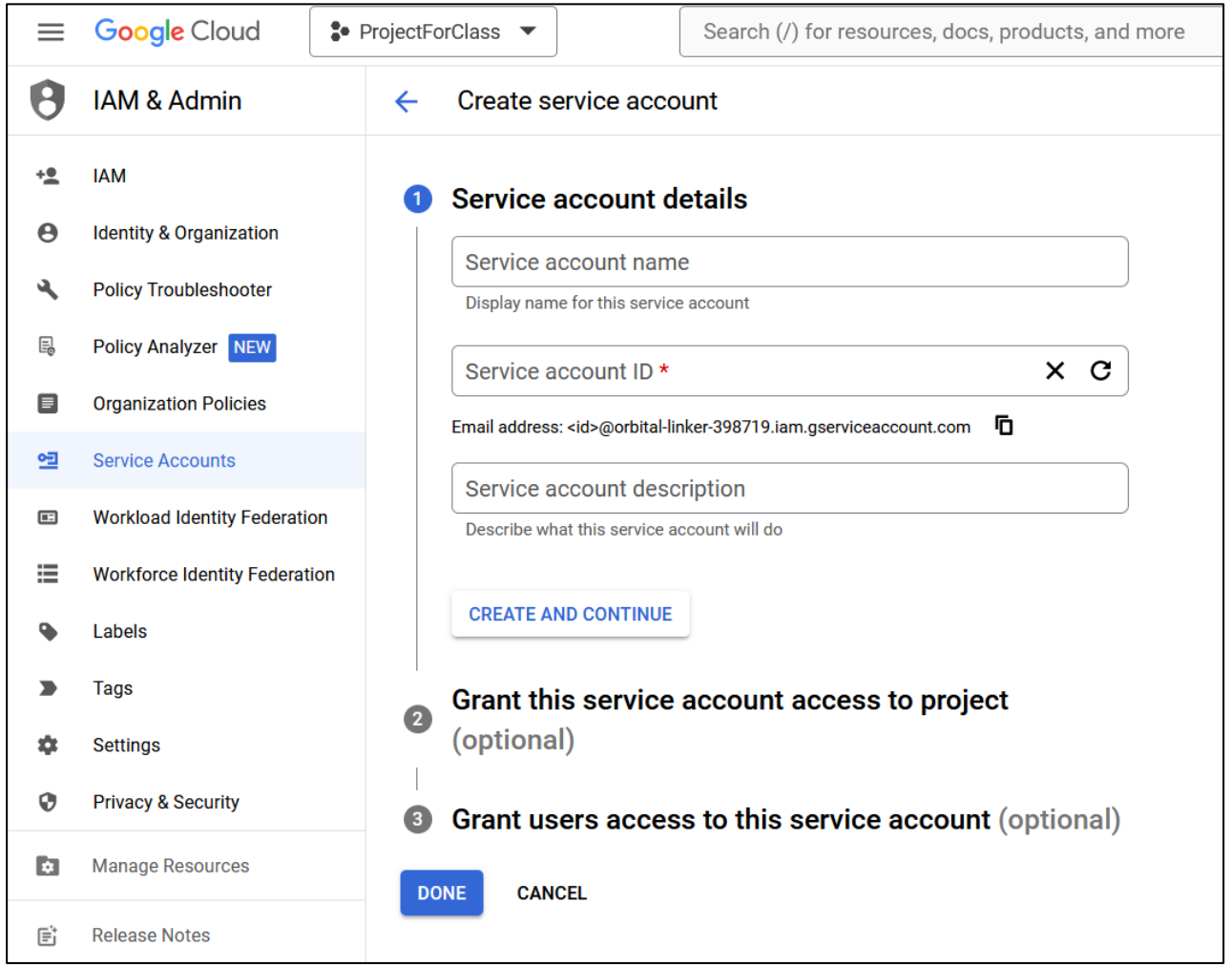
Service Account

- A service account is a special kind of account typically used by an application or compute workload, such as a Compute Engine instance, rather than a person.

https://cloud.google.com/iam/docs/service-account-overview

Walkthrough Creating a Service Account (1)

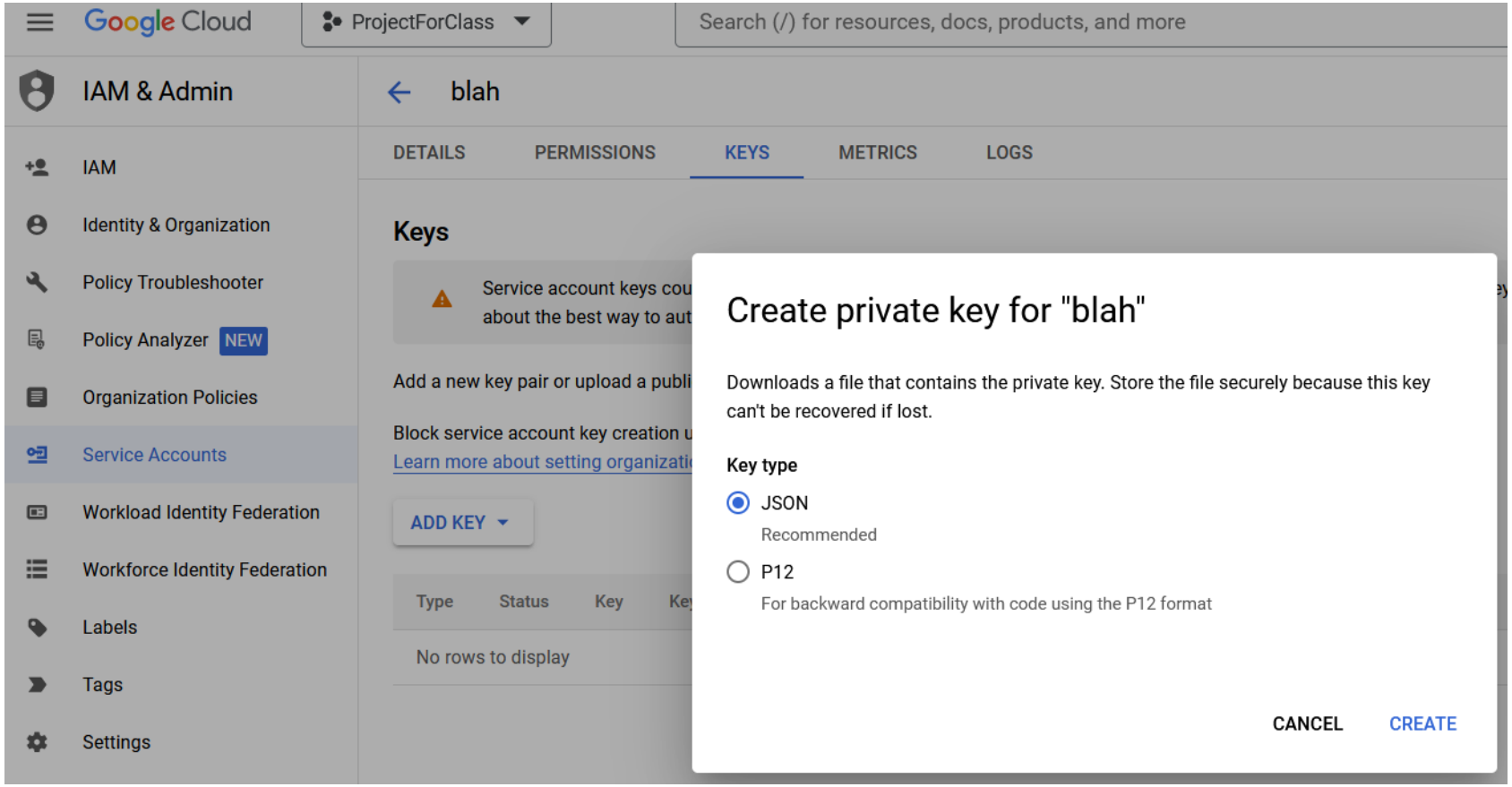




Role: Compute Engine Service Agent

Walkthrough Creating a Service Account (2)

- Create Key



Show Terraform Code

- Note – references the json file download

https://github.com/eric-keller/npp-cloud/tree/main/tf-mod1-demo

Demo Walkthrough

- Go into console and see VMs.
- terraform init
- terraform plan
- terraform apply
- Go into console and see VMs.



University of Colorado **Boulder**